



NORTHWESTERN UNIVERSITY

Computer Science Department

Technical Report
Number: NU-CS-2026-35

March, 2026

PYAC: Implementation of Algebraic Smooth Occluding Contours in Python

Kevin Ha

Abstract

Despite the maturation of the non-photorealistic rendering field over the past decades, instability across changing view directions has persisted in mesh-based line art algorithms. This complicates the rendering pipeline for users such as researchers and technical artists; both of whom take heuristics to fix such issues that are either computationally expensive or do not fully address the instability problem. Algebraic Smooth Occluding Contours addresses this instability issue through its use of a smooth surface construction of the underlying 3D mesh for which curves are calculated upon. However, even with this state-of-the-art approach providing topologically-valid smooth occluding contours that address this instability issue, its implementation proves difficult to work with, bringing its own issues regarding both approachability and replicability that prevent its widespread adoption. Therefore, I propose Python Algebraic Contours, which improves upon the aforementioned work by implementing it as a pip-installable Python library that is approachable for users like technical artists and addressing significant bugs in the original implementation that hinders its overall replicability for researchers. Despite its slower performance, my tool creates a way for researchers and technical artists to work with a state-of-the-art line art algorithm and paves the way for its future research.

Keywords

Python Algebraic Contours, Algebraic Smooth Occluding Contours, computer graphics, non-photorealistic rendering, line art, occluding contours, replication study

NORTHWESTERN UNIVERSITY

PYAC: Implementation of Algebraic Smooth Occluding Contours in Python

A DISSERTATION

SUBMITTED TO THE GRADUATE SCHOOL

IN PARTIAL FULFILLMENT OF THE REQUIREMENTS

for the degree

MASTER OF SCIENCE

Field of Computer Science

By

Kevin Ha

EVANSTON, ILLINOIS

March 2026

ABSTRACT

Despite the maturation of the non-photorealistic rendering field over the past decades, instability across changing view directions has persisted in mesh-based line art algorithms. This complicates the rendering pipeline for users such as researchers and technical artists; both of whom take heuristics to fix such issues that are either computationally expensive or do not fully address the instability problem. Algebraic Smooth Occluding Contours addresses this instability issue through its use of a smooth surface construction of the underlying 3D mesh for which curves are calculated upon. However, even with this state-of-the-art approach providing topologically-valid smooth occluding contours that address this instability issue, its implementation proves difficult to work with, bringing its own issues regarding both approachability and replicability that prevent its widespread adoption. Therefore, I propose Python Algebraic Contours, which improves upon the aforementioned work by implementing it as a pip-installable Python library that is approachable for users like technical artists and addressing significant bugs in the original implementation that hinders its overall replicability for researchers. Despite its slower performance, my tool creates a way for researchers and technical artists to work with a state-of-the-art line art algorithm and paves the way for its future research.

Keywords: Python Algebraic Contours, Algebraic Smooth Occluding Contours, computer graphics, non-photorealistic rendering, line art, occluding contours, replication study

ACKNOWLEDGMENTS

I would like to express my gratitude for advisor Professor Dietrich Geisler and their guidance in not only my Master's thesis, but also my journey across research, computer graphics, and software engineering; helping facilitate my growth as a software engineer and overall a better version of myself than the months and years before.

Committee members Professor Ian Horswill and Professor Emma Alexander also have my thanks for ensuring this research is of utmost quality. Thank you to Professor Philip Hayek, Professor Vincent St-Amour, and Instructor Andrew McGady because their recommendations allowed me to continue this half-decade educational journey. Thank you to Professor Gonzalo Bello and Professor George Maratos for their guidance as I began my computer science journey. And thank you to the other wonderful instructors I have had the pleasure of learning from.

Thank you to communities such as those centered around the Blender 3D modeling software and the broader non-photorealistic rendering field for sparking my initial interest in computer graphics nearly a decade ago and in making such non-photorealistic rendering research available for all. Additionally, the Northwestern Writing Place and my dear friend Jimmy Doan both have my gratitude for taking the time out of their day in making this dissertation possible and approachable. Additionally, I would like to thank my family for supporting me along my career path and my various ventures. Finally, thank you to all the friends I have made along this journey and the bonds forged and fostered along the way.

TABLE OF CONTENTS

Acknowledgments	2
List of Tables	8
List of Figures	10
Chapter 1: Introduction	13
1.1 Context	13
1.1.1 Media	14
1.1.2 Tooling	16
1.2 Motivation	17
1.2.1 Replicability	18
1.2.2 Approachability	19
1.2.3 Literature	19
1.3 Contribution	19
Chapter 2: Background	21
2.1 Occluding Contours	22
2.1.1 Mesh-based Approach	23
2.1.2 Smooth Approach	24

Chapter 3: Implementation	27
3.1 Design Philosophy	27
3.1.1 Translation and Localization	28
3.1.2 Approachability	28
3.1.3 Scope	29
3.2 Pipeline Overview	30
3.2.1 Mesh Input	30
3.2.2 Surface Construction	31
3.2.3 Occluding Contour Calculation	31
3.3 Translation	33
3.3.1 Direct Dependencies	34
3.3.2 Inkscape	35
3.3.3 Symbolic Mathematics	36
3.4 Localization	36
3.4.1 Shape Checking	37
3.4.2 Array Accessing	37
3.5 Test Suite	38
3.5.1 Pytest	38
3.5.2 Serialization	38
3.5.3 Branches	38
3.6 Limitations	39
3.6.1 Translation Versus Localization	39
3.6.2 Dependencies	39

3.6.3	Testing	40
Chapter 4:	Evaluation	41
4.1	Scope	41
4.2	Pipeline	42
4.2.1	Modifications	43
4.2.2	Normalization	43
4.2.3	Dataset	44
4.2.4	Tooling	44
4.3	Validity	45
4.3.1	Quantitative Results	45
4.3.2	Qualitative Results	46
4.4	Performance	47
4.4.1	Recursion Failure	47
4.4.2	Quantitative Results	48
4.4.3	Qualitative Results	49
4.5	Overall	49
4.5.1	Limitations	50
4.5.2	Dependent and Independent Views	51
4.5.3	Assertion False Positives	51
Chapter 5:	Discussion	52
5.1	Alternative Works	52
5.2	Tooling	52

- 5.3 Validity Theory 52
- 5.4 GPU Approaches 53
- 5.5 Machine Learning 53

- Chapter 6: Conclusion 54**
- 6.1 Future Work 54
 - 6.1.1 Urgent Fixes 55
 - 6.1.2 Longer-term Research 55

- References 60**

- Appendix A: Qualitative Evaluation 62**

- Appendix B: Quantitative Evaluation 70**

LIST OF TABLES

3.1	ASOC C++ packages and PYAC Python equivalents.	34
A.1	Explanation of colors in pixel-wise difference figures.	62
A.2	Car affine manifold, algebraic smooth surface, raster full occluding contours, vector full occluding contours, and vector visible occluding contours using chaining Quantitative Invisibility (left to right) (ASOC (top) vs PYAC (middle) vs difference (bottom)) (see Table A.1 for color map) (Car ©Pixar).	63
A.3	Killeroo affine manifold, algebraic smooth surface, raster full occluding contours, vector full occluding contours, and vector visible occluding contours using chaining Quantitative Invisibility (left to right) (ASOC (top) vs PYAC (middle) vs difference (bottom)) (see Table A.1 for color map) (Killeroo ©headus.com.au.).	64
A.4	Raster affine manifold using chaining Quantitative Invisibility (ASOC (left) vs PYAC (middle) vs difference (right)) (see Table A.1 for color map) (Bigguy and Monster Frog ©Bay Raitt. Fertility from AIM@SHAPE-VISIONAIR Shape Repository. Public domain Blub, Bob, and Ogre models by Keenan Crane. Open source Pawn model from OpenSubdiv.).	65
A.5	Raster smooth surface and visible occluding contours using chaining Quantitative Invisibility (ASOC (left) vs PYAC (middle) vs difference (right)) (see Table A.1 for color map) (Bigguy and Monster Frog ©Bay Raitt. Fertility from AIM@SHAPE-VISIONAIR Shape Repository. Public domain Blub, Bob, and Ogre models by Keenan Crane. Open source Pawn model from OpenSubdiv.).	66
A.6	Raster full occluding contours using chaining Quantitative Invisibility (ASOC (left) vs PYAC (middle) vs difference (right)) (see Table A.1 for color map) (Bigguy and Monster Frog ©Bay Raitt. Fertility from AIM@SHAPE-VISIONAIR Shape Repository. Public domain Blub, Bob, and Ogre models by Keenan Crane. Open source Pawn model from OpenSubdiv.).	67

A.7 Vector full occluding contours using chaining Quantitative Invisibility (ASOC (left) vs PYAC (middle) vs difference (right)) (see Table A.1 for color map) (Bigguy and Monster Frog ©Bay Raitt. Fertility from AIM@SHAPE-VISIONAIR Shape Repository. Public domain Blub, Bob, and Ogre models by Keenan Crane. Open source Pawn model from OpenSubdiv.). 68

A.8 Vector visible occluding contours using chaining Quantitative Invisibility (ASOC (left) vs PYAC (middle) vs difference (right)) (see Table A.1 for color map) (Bigguy and Monster Frog ©Bay Raitt. Fertility from AIM@SHAPE-VISIONAIR Shape Repository. Public domain Blub, Bob, and Ogre models by Keenan Crane. Open source Pawn model from OpenSubdiv.). 69

B.1 Statistics for quantitative validity results. Note that these values do not include the failed "car" mesh in their calculation. Differences are calculated from the mean of differences across all 30 successful meshes out of the 31 total evaluated. 72

B.2 Statistics table of runtime performance results throughout the end-to-end occluding contours pipeline. Note that slowdown is calculated from the mean of all slowdowns throughout the 30 successful meshes out of the 31 total evaluated. AS is shorthand for ASOC. PY is shorthand for PYAC. As denoted by the asterisk (*), values calculated utilizing the "car" mesh runtime results. 74

B.3 Raster full occluding contours showcasing ASOC's instability in its occluding contour visibility calculation using the propagation Quantitative Invisibility method (ASOC (left) vs PYAC (middle) vs difference (right)) (see Table A.1 for color map) (Public domain Bob, Blub, Spot, Pig, and Ogre models by Keenan Crane.). . 75

B.4 Vector full occluding contours showcasing ASOC's instability in its occluding contour visibility calculation using propagation Quantitative Invisibility. (ASOC (left) vs PYAC (middle) vs difference (right)) (see Table A.1 for color map). (Public domain Bob, Blub, Spot, Pig, and Ogre models by Keenan Crane). 76

LIST OF FIGURES

1.1	Girls Band Cry (2024) ©Toei Animation	15
1.2	Patlabor EZY (2026) ©HEADGEAR	15
1.3	NPR Quake (2004) (Figure from [22].)	16
1.4	Return of the Obra Dinn (2018) ©Lucas Pope	17
2.1	Smooth occluding contours with all segments and dots representing node information such as cusps (left) and visible segments only (right) (Public domain Spot model by Keenan Crane. Figure from [11]).	22
2.2	Mesh-based occluding contours (Figure adapted from [42]).	23
2.3	Smooth occluding contour visualized for perspective projections utilizing camera position $\mathbf{c}$ (Figure adapted from [45]).	24
2.4	UV domain curve maps onto smooth patch in the case after retrieving an equivalent orthographic projection of the mesh and $\tau = [0, 0, 1]$. (Public domain Spot model by Keenan Crane. Figure from [11]).	25
2.5	Mesh after UV unwrapping (Public domain Spot model by Keenan Crane. Figure from [11]).	26
2.6	Vertices under orthographic projection (a) versus perspective projection (b). In ASOC's implementation, $\tau = [0, 0, 1]$ represents the view direction. (Figure from [42]).	26
3.1	ASOC smooth surface fit (weaker fit left to stronger fit right) (Figure from [11]). . .	29
3.2	PYAC end-to-end occluding contour generation pipeline. Note that the 3D mesh input also includes its UV coordinates generated from the process in [35]. (Public domain Spot model by Keenan Crane)	30

3.3	Interior cusps within surface patches and boundary cusps between patch boundaries (Figure from [11]).	32
3.4	Cones denoted by the green vertices b . The specifics for cones are explored in [11] (Figure from [11]).	33
3.5	PYAC Polyscope wire screenshot in 3D space to showcase the line being single, closed form loop.	33
B.1	Smooth surface runtime of ASOC (blue) versus PYAC (red) in seconds.	71
B.2	Slowdown of each step within the occluding contour calculation itself with the occluding contour constructor call to showcase scale relative to the rest of the process.	72
B.3	Slowdown of each step within the occluding contour calculation itself without the occluding contour constructor call to showcase scale relative to the rest of the process. See Figure B.2 for the full figure including the occluding contour constructor call.	73
B.4	Performance of various steps within the occluding contour calculation itself, with the occluding contour constructor call slowdown greatly outweighing the rest of the parts in the occluding contour calculation.	77
B.5	Chart of the number of faces for all 31 meshes used for evaluation. Both ASOC and PYAC read in the same number of triangle faces. Note that all plots are sorted by increasing number of faces from smallest to largest.	78
B.6	Chart of the number of intersection calls by mesh for ASOC.	79
B.7	Chart of the number of patches for the smooth surface of each mesh. Here, same-ness in the number of patches represents the stability of the surface construction to noise. Note that the "car" mesh is missing its numerical data label for PYAC since its end-to-end occluding contour generation pipeline failed.	80
B.8	Chart of the number of segments per mesh. Note that the "car" mesh does not have an equivalent PYAC bar data label because its evaluation failed the propagation Quantitative Invisibility step.	81
B.9	Chart showing slowdown of PYAC smooth surface and patch boundary calculations.	82
B.10	ASOC total contour calculation time (seconds) vs. number of faces (triangles). Note that the "car" mesh does not have any metrics since it failed the propagation Quantitative Invisibility step.	83

B.11 PYAC total contour calculation time (seconds) vs. number of faces (triangles). Note that the "car" mesh does not have any metrics since it failed the propagation Quantitative Invisibility step.	84
--	----

CHAPTER 1

INTRODUCTION

1.1 Context

Computer graphics usually associates itself with photorealism [1], which it defines as qualities akin to that of photographs [2]. However, in the past decades, non-photorealistic rendering (NPR) has established itself within the broader computer graphics field; setting itself apart from its peers by aiming for the opposite of photorealism [3, 4]. In other words, NPR can be defined as processes that counter photorealism and opt for stylization instead. This NPR field has its subsets as well, with parts focusing on stylization using techniques such as **shading** [5, 6, 7] and **line art** [8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19], with the latter serving as the focus of this paper.

Within NPR’s line art subfield in particular, line types such as apparent ridges, suggestive contours, and especially **occluding contours** serve as lines of interest for research. **Occluding contours** separate front-facing faces from back-facing faces. However, mesh-based occluding contour implementations often encounter instability, with previous attempts having been costly in terms of performance [8, 14] or having visual artifacts within the contour lines themselves [20]. This problem is not only inherent to occluding contours, but also to other line types as well. Stability within the line art subfield of NPR is often coined as consistency in lines across different view directions. Terms such as flickering and inconsistency represent undesirable qualities that serve as the antithesis to stability [21]. Hence, solutions such as **Algebraic Smooth Occluding Contours** (ASOC) provide stable line art by generating its lines upon a smooth surface representation of an input 3D mesh rather than the 3D mesh data itself [11]. However, as discussed in Section 1.2, ASOC faces issues with both **approachability** and **replicability** that may explain its general lack of adoption.

As the progenitor of NPR line art, pen and pencil line artists utilize techniques such as hatching,

line speed, and line quality in crafting images, considering principles such as shape and form. However, such techniques are not needed for creating a photorealistic image. Rather, artists have freedom in coordinating their line strokes and emphasizing or omitting particular features. Such qualities in human-made line art translates to line art research as well in that both are the mapping of deliberately placed strokes of lines for forming images.

1.1.1 Media

NPR techniques may play pivotal roles in achieving a homogeneous visual style, which in this paper’s context is defined as a coherent visual aesthetic consistent with the rest of the work or piece of media. Some notable examples include *Girls Band Cry* (2024), whose stylized NPR shading and line art on its characters and environment are not only trying to achieve the usual Japanese animation aesthetic, but also managing to fit the expressive nature of the 2D animation format for its 3D characters as well (see Figure 1.1). Additionally, NPR techniques find themselves used for streamlining the animation process, allowing for rich, complex scenes while also accelerating the animation process by reducing the necessity for hand-drawn 2D frames. A more recent example showcasing this technique, *Patlabor EYZ* (2026) utilizes NPR shading on its 3D bipedal mechanized mobile suits with aims to homogenize its appearance and style with the rest of the 2D-drawn characters and environments while also allowing for dynamic camera angles and complex views (see Figure 1.2).

In interactive digital experiences where performance is of essence, more imprecise, yet stylized techniques are used. These works achieve their own unique visual style with inspiration from 2D media. One such game includes *NPR Quake* (2004) [22], which offers a variety of visual aesthetics matching that of technical design blueprints or notebook pencil sketches in the objects and environment themselves (see Figure 1.4). Taking cues from early twentieth century North American commercials, *Team Fortress 2* (2007) deliberately uses NPR techniques in its shaders, which coalesces with the rest of its chosen style for an overall coherent aesthetic [23]. *Return of the Obra Dinn* (2018) approaches its style with different NPR techniques, utilizing a 1-bit color



Figure 1.1: Girls Band Cry (2024) ©Toei Animation

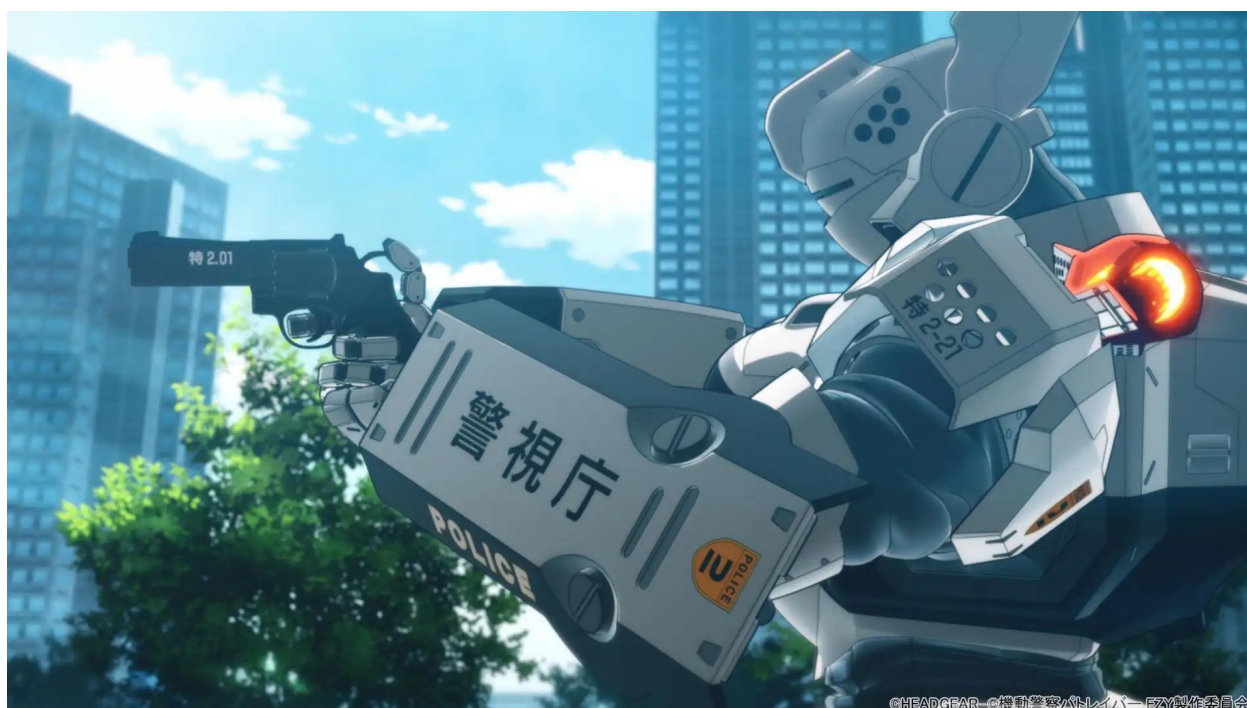


Figure 1.2: Patlabor EZY (2026) ©HEADGEAR

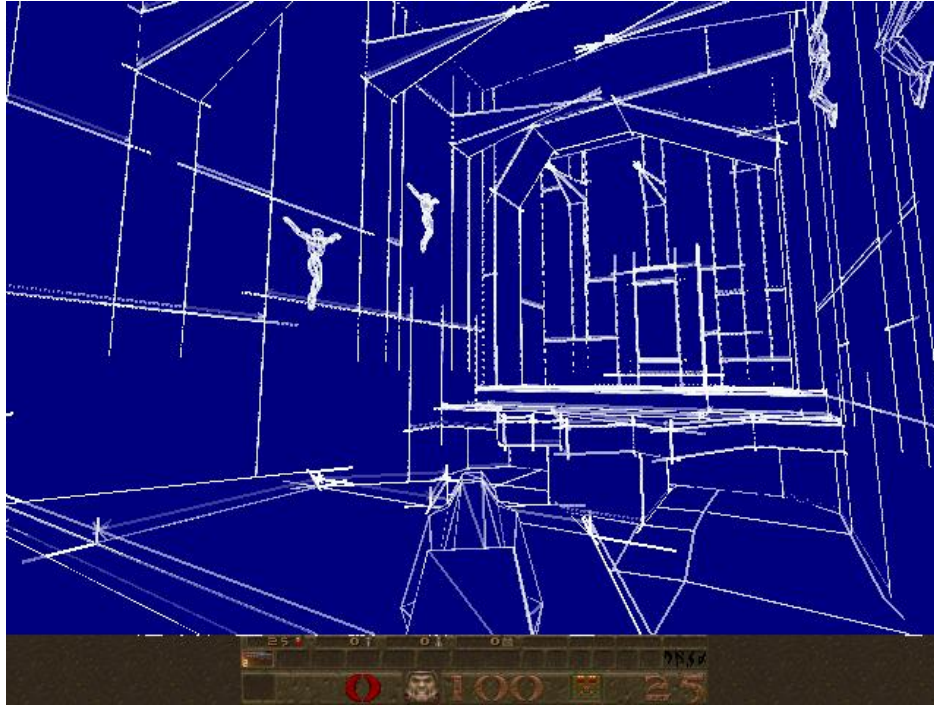


Figure 1.3: NPR Quake (2004) (Figure from [22].)

palette alongside dithering to achieve a style akin to that of games for the Apple Macintosh Plus (see Figure 1.4) [24, 25].

1.1.2 Tooling

Tooling exists in facilitating NPR in creative works as well, particularly in the 3D modeling program *Blender*, which this paper utilizes for its user friendliness and attainability, especially as its availability comes without monetary costs relative to its alternatives such as Maya. Within Blender and NPR line art in particular, the Blender FreeStyle render engine allows for programmable line art and features extensive documentation throughout its development process [26], aiding in making line art types such as ridges, valleys, and occluding contours more **approachable** to technical artists, albeit non-real-time. Following Blender Freestyle, the most recent addition to Blender is LANPR, otherwise known as Blender Line Art, which offers features such as real time line art and native integration with Blender’s Grease Pencil strokes as a foundation for its line art rendering [27, 28, 29]. Demand for such line art tools extends commercially as software such as Pencil+4

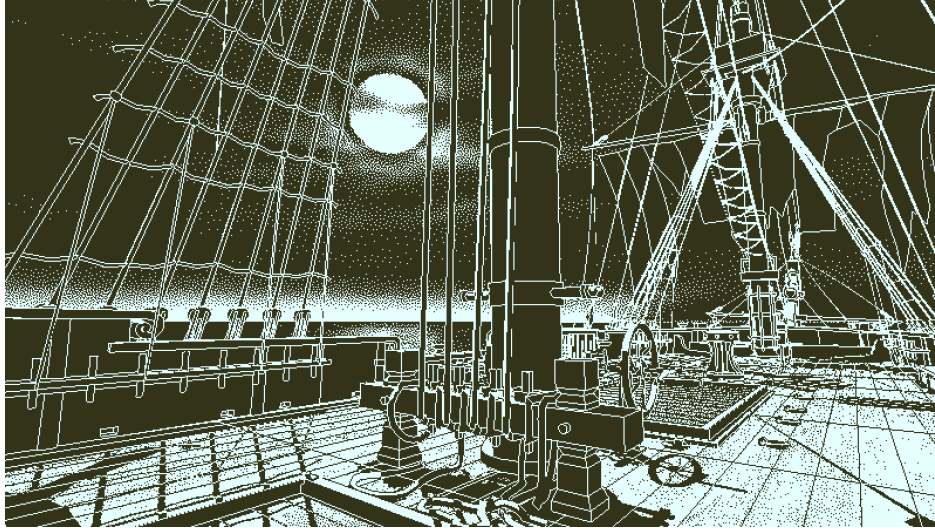


Figure 1.4: Return of the Obra Dinn (2018) ©Lucas Pope

features an entire suite of NPR tools, along with an independent line art renderer. In addition, for NPR in general rather than specifically line art, efforts are also ongoing to improve the NPR *workflow* within Blender. This entails the Blender Extender Expressive Renderer’s successor Malt, which provides a Blender rendering add-on specifically for NPR workflows and programmable shaders utilizing the OpenGL Shading Language [30]. This incentive for NPR is more apparent with Blender’s proposed NPR branch, which aims towards an NPR workflow integrated within Blender itself and improves functionality in features such as shading and compositing [31, 32].

1.2 Motivation

These advancements and utilization of NPR in various media and tooling have not caught up with the rest of the computer graphics field, particularly in the case of **replicability** and **approachability**. ASOC especially faces these issues, which ultimately stunts the adoption of its state-of-the-art techniques in fixing the line stability issue. In the upcoming subsections, I will explore the greater issue of replicability and approachability with ASOC and how my tool Python Algebraic Contours (PYAC) builds upon ASOC’s novel state-of-the-art algorithm for users such as researchers and technical artists.

1.2.1 Replicability

Despite advances in the field of computer graphics, replicability remains an issue; however, recent efforts advocate for improved research access to computer graphics itself [33, 34]. Various challenges in replicability were encountered in PYAC’s implementation of ASOC as well. Upon initially installing ASOC, dependencies such as the Intel Math Kernel Library (MKL) were no longer available from their linked URLs, meaning that ASOC’s CMake files were left broken. Additionally, ASOC’s build process proved incompatible with operating systems such as Microsoft Windows. Also, while evaluating ASOC in a virtual machine, dependencies for CMake and GCC version were not specified, leading to increased time spent resolving such issues.

There are also replicability concerns regarding the ASOC researchers’ approach to their implementation. Specifically, their approach is highly dependent on the UV unwrapping process provided in [35]. Therefore, it is unclear how generalizable ASOC’s occluding contour calculation pipeline is across different manifold meshes other than the ones that the ASOC researchers included in their study. Additionally, the ASOC researchers performed preprocessing to remove intersections and very short edges from their mesh dataset. However, it is unclear to what extent this impacted the performance or output of their occluding contour algorithm.

Throughout the implementation of PYAC, further concerns with ASOC’s replicability arose. One such issue involved the extent to which their algorithm must guarantee mathematical correctness since there are many assertions within the ASOC codebase, particularly in the categorization of conics, that trigger despite the output occluding contour image appearing valid. Additionally, another issue was brought about by the provided camera matrices with the ASOC dataset because it was discovered that these extrinsic camera matrices also performed scaling and that any naive input of a camera matrix with only its rotation and translation would result in occluding contours that are out-of-bounds of the viewport. Finally, ASOC’s implementation of particular optimizations, such as its propagation Quantitative Invisibility method, resulted in invalid output that differs from its more stable chaining Quantitative Invisibility method. These issues are discussed later in the paper but are worth mentioning now as they proved detrimental to ASOC’s replicability.

1.2.2 Approachability

Approachability encompasses quality-of-life features such as clear variable naming, coherent structure and relationships between components. This transcends software engineering and goes towards general understandability of the codebase for people ranging from entry-level computer scientists to seasoned computer graphics professionals. This approachability is not only for the developers working on PYAC, but also for its users, which include technical artists who want to utilize such novel topologically-valid occluding contours. For example, approachability can be seen in the context of approaching a house cat versus a mountain lion; hopefully, one is less intimidating and overwhelming than the other. Overall, approachability is defined as **ease of access, setup, use, and readability**. It is not a definitive metric by any means but is interconnected with **replicability**.

1.2.3 Literature

Since ASOC’s debut at SIGGRAPH 2023, the literature utilizing its novel approach has been sparse. Adjacent tools such as ApparentRidges offer flexibility via its Haxe implementation, allowing its compilation into programming languages such as Python. Its developers have provided documentation and various use-case examples to aide in approachability, but ApparentRidges [36] ultimately relies on an approach to contour – not necessarily occluding contour – generation that suffers from instability. Blender Freestyle helps artists and even provides data for research [37]. However, its implementation is reliant on mesh-based approaches that are susceptible to instability across view directions and are generally catered towards artists and not researchers, though its original non-Blender implementation is available for use [10].

1.3 Contribution

Python Algebraic Contours (PYAC) addresses ASOC’s approachability problem via a representation removed from the major pains of the usual software engineering hassles. Such goals are

achieved with the shift away from ASOC’s C++ application and towards a Python library, creating a familiar environment for users such as technical artists experienced with Blender’s Python add-on API. Therefore, akin to libigl’s contribution to computational geometry, Pytorch’s contribution to deep learning, or librosa’s contribution to audio processing, PYAC aims to contribute to the overarching line art literature. Additionally, through approachability, PYAC aims for interoperability in applications as well, as showcased in its accompanying proof-of-concept add-on *Ranato*.

PYAC provides the following contributions for **approachability**:

- Pip-installable Python library, making the tool accessible to users such as researchers and technical artist.
- Replaces C++ libraries with common Python counterparts. Such an example is Eigen and its Intel MKL dependency and its counterpart NumPy, a well-documented and popular library with stable long-term support.
- Clear installation instructions with explicit CMake version requirements and dependency specifications.

PYAC also provides the following contributions for **replicability**:

- Docker container available for reproducing the results of this paper.
- Documents dependencies and addresses bugs inherited from the original ASOC implementation.
- Reduces reliance on fragile or potentially unavailable dependencies such as Intel MKL.
- Validates stability of the line art algorithm across ASOC’s dataset meshes.

Utilizing Blender’s revamped Python API package manager via Python wheels, as introduced in Blender 4.2, PYAC has been implemented as a Blender 4.5 add-on called *Ranato*. *Ranato* is implemented in Python 3.11.9 to coincide with Blender 4.5 LTS’ Python environment [38]. Docker images for PYAC and ASOC are available as well. PYAC’s source code is available at [39].

CHAPTER 2

BACKGROUND

Various literature reviews have captured the broader NPR landscape [21, 40], with others focusing specifically on line art [41, 42, 43] and provide detailed insight into the mathematical foundation for such research as well. This chapter frequently utilizes explanations and figures of occluding contours from [42], specifically Chapter 3 for its occluding contour definition, Chapter 4 for its visibility calculation, and Chapter 7 for its smooth occluding contour explanation. Particular works clarify the distinction between different line types [41], though PYAC focuses specifically on occluding contours in correspondence with ASOC. However, as discussed in Section 6.1, PYAC can be extended to alternative line types. It should be noted that the surrounding literature often references occluding contours as simply contours or silhouettes [8]. For consistency, I utilize the term **occluding contours** for its greater clarity over the aforementioned terms. Also, the upcoming explanation refers to both 2D and 3D curves as occluding contours. Additionally, the ASOC paper refers to the smooth surface construction of the mesh as the piecewise-quadratic Powell-Sabin surface, the algebraic surface, and the smooth surface. For simplicity, I utilize the term **smooth surface**. It is also worth noting that smooth surfaces consist of patches similar to how meshes consist of faces. Simply put, patches are like faces but for smooth surfaces. Finally, the ASOC paper refers to the UV unwrapping from [35] as the conformal parametrization, mesh parametrization, and the parametric domain. For simplicity, I utilize the term **UV unwrapping**. With these in mind, the upcoming section provides a high-level overview of the occluding contour algorithm that ASOC implements. Though, for the sake of scope, topics from the ASOC paper such as thin-plate optimization and its various Quantitative Invisibility techniques are excluded from this discussion.

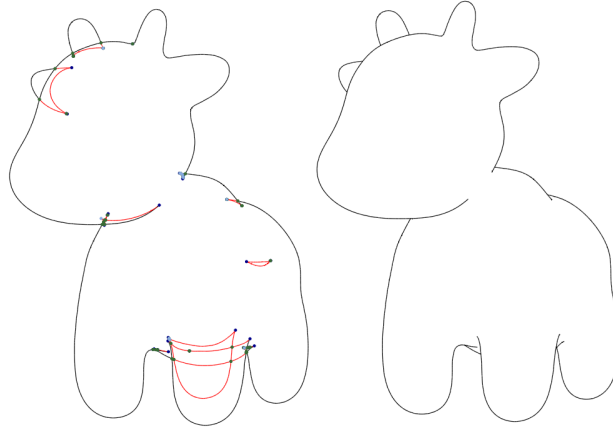


Figure 2.1: Smooth occluding contours with all segments and dots representing node information such as cusps (left) and visible segments only (right) (Public domain Spot model by Keenan Crane. Figure from [11]).

2.1 Occluding Contours

The approaches in ASOC and therefore PYAC assume that meshes are **manifold** and **orientable**. Here, **orientability** is defined such that a mesh has consistent normals for nearby groups of inward and outward-facing faces. In the case of mesh-based occluding contours, these *polylines* are defined such that they separate front-facing surfaces from back-facing surfaces from a given camera direction. For smooth occluding contours, *curves* separate front and back-facing surfaces rather than polylines. These smooth occluding contours differ from simple outlines of a mesh in that they capture the features within the mesh as well, not just its perimeter.

After calculating the occluding contours for a mesh, it is important to determine the parts of the occluding contours that are visible from the current view direction via the Quantitative Invisibility algorithm [44]. While ASOC’s paper covers its implementation of the Quantitative Invisibility algorithm, PYAC does not explore this in detail for the sake of scope, but Figure 2.1 showcases the state of the occluding contours after this visibility calculation has been performed. For more information about Quantitative Invisibility and its relation to ray tests, Chapter 4 of [42] can be referenced. For simplicity, this visibility step entails splitting occluding contours into **segments** and performing boundary checks of the occluding contours relative to the patches of the smooth surface and to one another.

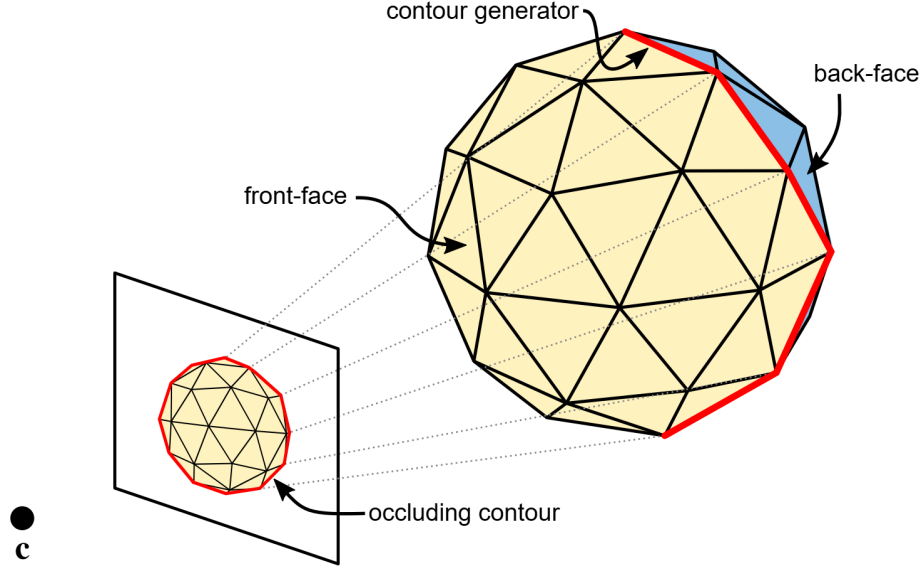


Figure 2.2: Mesh-based occluding contours (Figure adapted from [42]).

2.1.1 Mesh-based Approach

This subsection utilizes explanations provided in Chapter 3 of [42] and assumes a triangulated mesh and a perspective camera. Here, $\mathbf{c}$ is defined as the camera position in the world space. The position $\mathbf{p}$ is that of a triangular face's vertices where $\mathbf{p} = [p_x, p_y, p_z]^T$ in 3D space. The view direction $\mathbf{v}$ is such that $\mathbf{v} = (\mathbf{c} - \mathbf{p})$. The face normal $\mathbf{n}$ is dependent on the 3D position of its vertices and is defined by $\mathbf{n} = (\mathbf{p}_3 - \mathbf{p}_1) \times (\mathbf{p}_2 - \mathbf{p}_1)$. With the variables defined, front-facing faces are defined as $(\mathbf{c} - \mathbf{p}) \cdot \mathbf{n} > 0$ and back-facing faces are defined as $(\mathbf{c} - \mathbf{p}) \cdot \mathbf{n} < 0$ (see Figure 2.2). However, in the case where an edge directly faces the camera (i.e. $(\mathbf{c} - \mathbf{p}) \cdot \mathbf{n} = 0$), the implementation is left to the programmer. Factors such as noise inherent to floating point calculations can inflict instability on the occluding contours, causing the calculated occluding contours to flicker between the front-facing case and the back-facing case across different view directions. Therefore, it would be advantageous to simply represent the occluding contours as a closed-form equation such as $(\mathbf{c} - \mathbf{p}) \cdot \mathbf{n} = 0$ rather than relying on the aforementioned front-facing and back-facing inequalities.

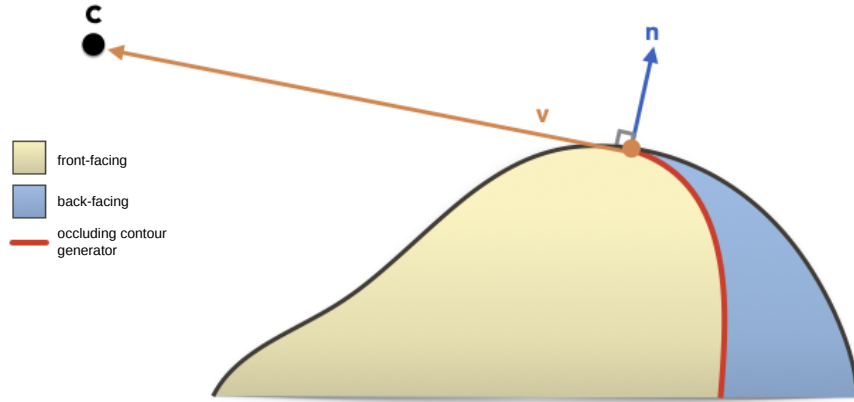


Figure 2.3: Smooth occluding contour visualized for perspective projections utilizing camera position $\mathbf{c}$ (Figure adapted from [45]).

2.1.2 Smooth Approach

This subsection utilizes Chapter 7 of [42] for its explanation. For smooth occluding contours, there exists a native algebraic representation of the smooth surface that provides a closed-form equation of the occluding contours (i.e. $(\mathbf{c} - \mathbf{p}) \cdot \mathbf{n} = 0$). Therefore, instead of referring to the curves as occluding contours, they instead represent **occluding contour generators**. Unlike the mesh-based approach that is dependent on a discrete set of vertex positions, smooth occluding contours are represented by algebraic curves that can then be sampled infinitely (see the red curve in Figure 2.3). In this smooth approach, the occluding contours instead refer to the curve formed by points sampled from the occluding contour generator (e.g. the orange dot in Figure 2.3) that is then projected onto the image plane, similar to the projection seen in Figure 2.2.

From this foundation, ASOC utilizes a UV unwrapping step from [35] that provides a mapping from the 3D mesh to the patches of the smooth surface (see visualizations in Figure 2.4 and Figure 2.5). Both [35] and [11] cover the theory behind this UV unwrapping and how its features prove critical to finding the mapping from the 3D mesh to the smooth surface patches, but are otherwise out of the scope of this paper.

The occluding contour calculation starts from the 2-dimensional ($\mathbb{R}^2$) $[u, v]$ domain with a parametric curve $\mathbf{r}(t)$. The coordinate system of $\mathbf{r}(t)$ is represented by $\mathbf{r} \equiv [u, v]$. The parametric

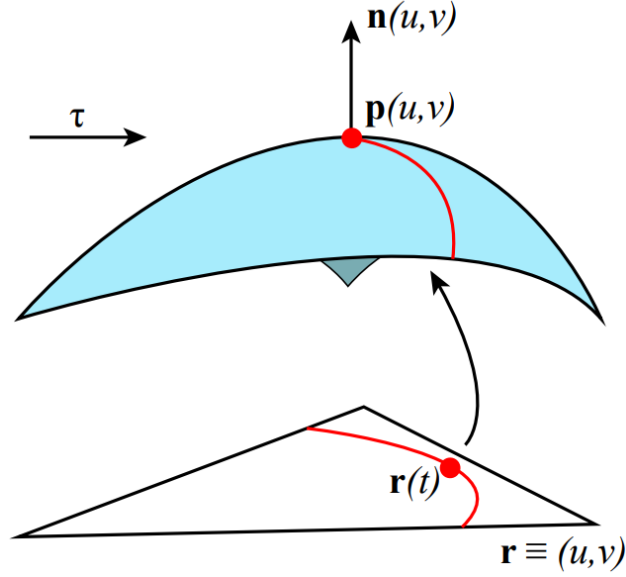


Figure 2.4: UV domain curve maps onto smooth patch in the case after retrieving an equivalent orthographic projection of the mesh and $\tau = [0, 0, 1]$. (Public domain Spot model by Keenan Crane. Figure from [11]).

curve $\mathbf{r}(t)$ can then be mapped into the 3-dimensional ($\mathbb{R}^3$) domain of the smooth surface patch $\mathbf{p}(u, v)$ with the composition $\mathbf{p}(\mathbf{r}(t))$. Within the 3D domain, the normal $\mathbf{n}$ is defined by $\mathbf{n}(u, v) = \mathbf{p}_u \times \mathbf{p}_v$, which can be of non-unit length [11]. Finally, $\mathbf{c}$ represents the perspective camera position in 3D space with $\mathbf{P}$ representing the image plane. Overall, the UV coordinate system is used to find the location of the algebraic curves on the smooth surface patches for the occluding contour equation $\mathbf{n}(u, v) \cdot (\mathbf{c} - \mathbf{p}(u, v)) = 0$

The occluding contour equation can then be simplified to $\mathbf{n}(u, v) \cdot \tau = 0$ under an equivalent orthographic projection since it removes the need for the camera position $\mathbf{c}$ (as visualized in Figure 2.6). Now, $\tau = [0, 0, 1]$ is the view vector looking straight towards the patches of the smooth surface under an equivalent orthographic projection. Alternatively, this can be seen as taking a "screenshot" of the mesh into a different representation. Therefore, any different value of τ would break that illusion and the equivalent orthographic projection is no longer valid. This intuition can also be explored by viewing the occluding contours in 3D space using PYAC.

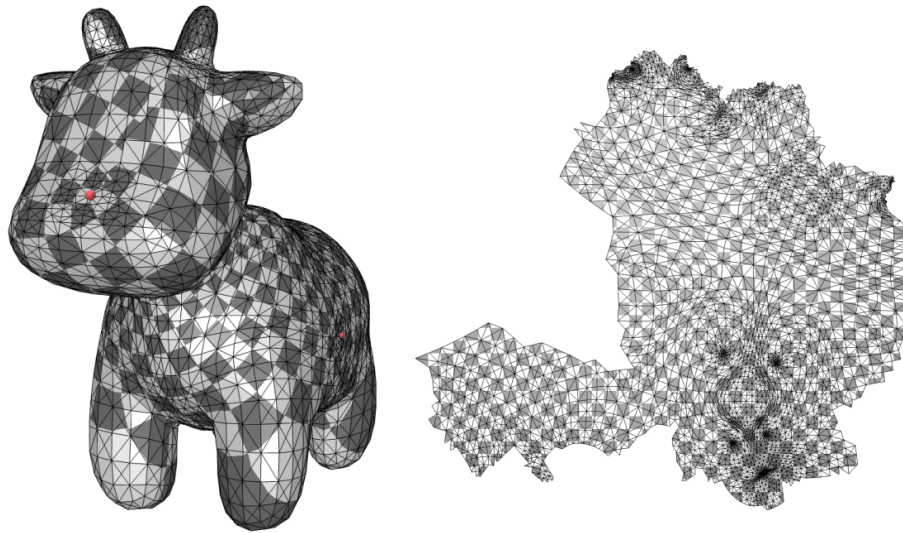


Figure 2.5: Mesh after UV unwrapping (Public domain Spot model by Keenan Crane. Figure from [11]).

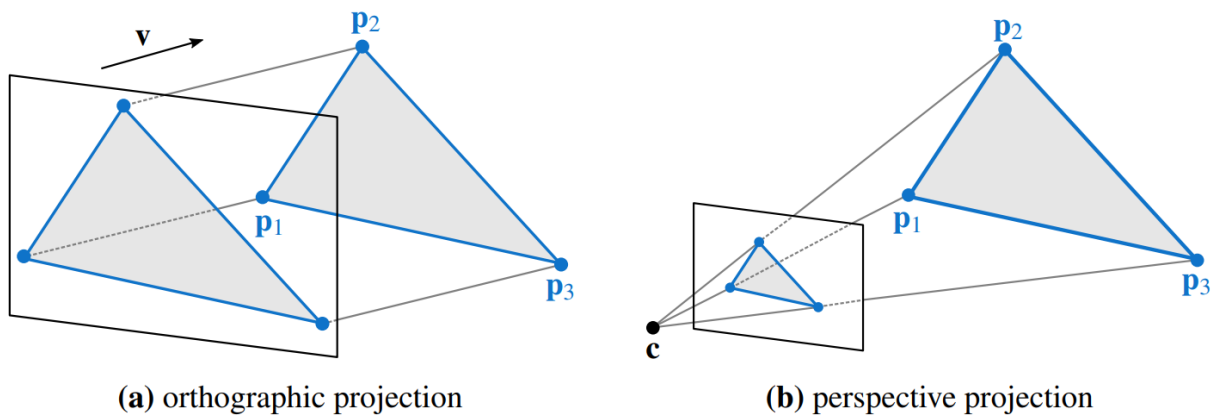


Figure 2.6: Vertices under orthographic projection (a) versus perspective projection (b). In ASOC's implementation, $\tau = [0, 0, 1]$ represents the view direction. (Figure from [42]).

CHAPTER 3

IMPLEMENTATION

This upcoming chapter focuses on the major structural code implementation of PYAC and design decisions taken when porting ASOC’s C++ application to PYAC’s Python library. This helps highlight the technical debt that PYAC inherits from ASOC but also the improvements that PYAC offers for approachability and replicability. Despite ending as a Python library, PYAC originates from a direct translation of ASOC and its end-to-end pipeline from 3D mesh input to 2D occluding contour output. Therefore, much of PYAC’s implementation reflects this origin, which extends to this upcoming chapter as well. Note that the ASOC paper utilizes terminology such as **conformal parametrization** and the **Powell-Sabin surface** with reference to the UV unwrapping step and the smooth surface, respectively. As mentioned previously, I use the latter terminology for simplicity. Additionally, the ASOC paper refers to the smooth surface as the quadratic spline surface. Also, the terminology *testing* and *debugging* are used throughout this chapter. Here, testing refers to the test suite itself and its utilization. Meanwhile, debugging refers to catching errors in the context of ensuring PYAC’s functionality matches that of ASOC.

3.1 Design Philosophy

This section explores the reasoning behind major design decisions in PYAC and its conversion from ASOC’s C++ application to a Python library. This helps clarify PYAC’s contribution beyond a simple re-implementation of ASOC and also establishes guidelines for changes to PYAC and potentially its future developments. This includes balancing **translation** and **localization** of ASOC’s original code, emphasizing **approachability** and **replicability** in PYAC’s code, and also the **scope** for PYAC’s development leading up to the writing of this paper.

3.1.1 Translation and Localization

The usage of the terms *localization* and *translation* originates from converting foreign languages from one *source* to its *destination* and the tradeoff between faithfulness to the source language and adaptation to the destination language. **Translation** preserves the nuance of the *source* language, but may not adapt well to the intricacies of the destination language. Meanwhile, **localization** adapts to the structure, functionality, and intricacies of the *destination* language, but may not accurately capture the *source* language’s nuance.

PYAC’s implementation reflects this language-based terminology. The aim of PYAC’s implementation lies in its faithfulness to ASOC as much as possible with reasonable exceptions. If translations from C++ to Python directly are unachievable, then alternative approaches for localization may be taken as long as the underlying functionality remains the same and is not harmful to the rest of the occluding contour generation pipeline. In other words, changes were not made unless they improved the quality-of-life for the user. This rationale is in part a result of ASOC’s delicate operations that are susceptible to differing logic or precision errors that not only affect the output occluding contours but also its intermediate calculations as well. Despite PYAC’s origins as an ASOC translation to ensure its functional correctness with its source, more consideration towards localization to Python is advised as PYAC matures.

3.1.2 Approachability

Another major philosophy guiding PYAC’s development lies in improving the **approachability** of ASOC’s occluding contour pipeline for users such as researchers and technical artists. Specifically, PYAC improves upon approachability through its implementation in Python, which allows for users such as technical artists familiar with Blender’s add-on API to easily integrate PYAC into their projects. Additionally, PYAC utilizes popular pip-installable libraries such as NumPy, which serves in contrast to ASOC’s Eigen library which suffered from its Intel MKL dependency becoming unavailable. PYAC’s libraries such as NumPy feature extensive community support, a plethora of documentation, and runs on different operating systems. Additionally, the availabil-

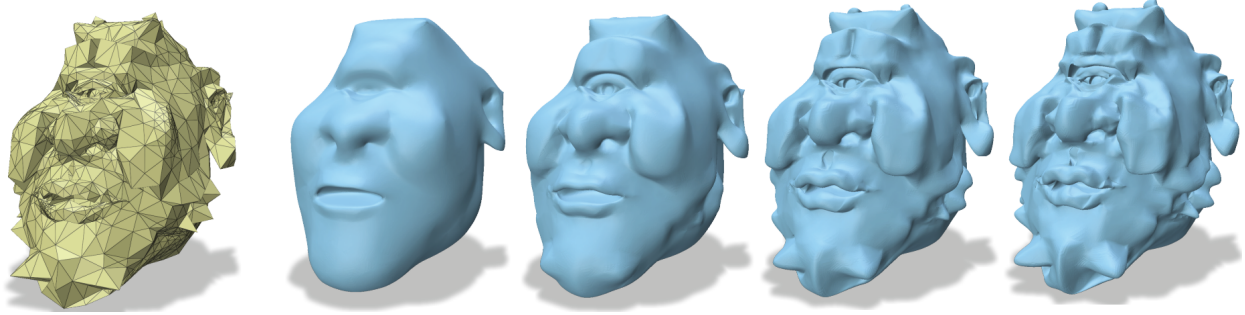


Figure 3.1: ASOC smooth surface fit (weaker fit left to stronger fit right) (Figure from [11]).

ity of Python bindings for popular geometry libraries such as *Polyscope* and *libigl* prove vital to PYAC’s functionality and approachability for researchers who utilize such libraries.

3.1.3 Scope

In terms of **scope**, PYAC focuses on implementing major components of ASOC’s end-to-end occluding contour generation pipeline. This scope provides reasoning behind the omissions of particular scripts and functionality from ASOC that no longer contribute to the end-to-end occluding contour generation pipeline. However, some functionalities that display or help provide geometric intuitions of ASOC’s underlying concepts prove worthy of re-implementation into PYAC’s suite of library functions in future iterations of PYAC.

In addition, ASOC focuses a part of its implementation on the optimization of the smooth surface to the underlying 3D mesh through a process known as the thin-plate optimization. This process determines the fit (i.e. the “shrink-wrapping”) of the smooth surface onto the 3D mesh (see Figure 3.1). For scope, this paper omits the aforementioned fit step from discussion as PYAC focuses on implementing ASOC’s core functionality, which is primarily in the smooth surface construction and occluding contour generation itself. Though, note that PYAC does implement this functionality, but its correctness has not been tested.

Additionally, ASOC relies on a UV unwrapping process from [35] that provides the necessary information for the entire occluding contour generation pipeline covered in Section 3.2. Practically, the ASOC authors performed this stage in a separate, external application outside of the

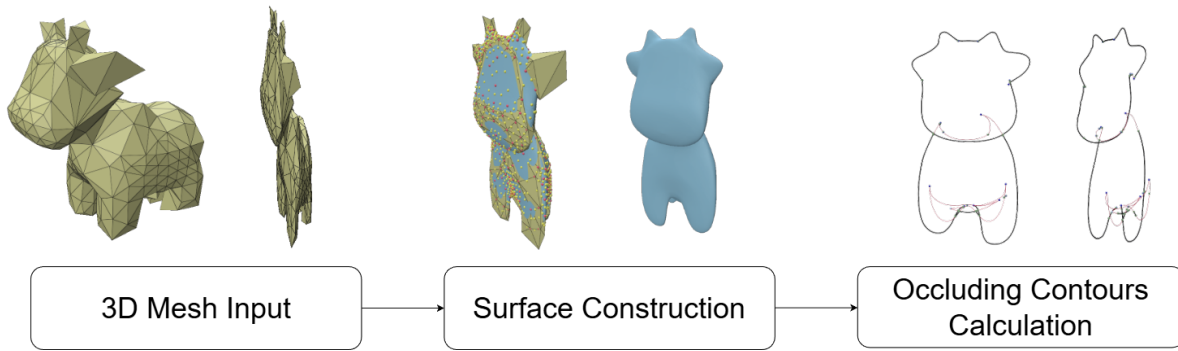


Figure 3.2: PYAC end-to-end occluding contour generation pipeline. Note that the 3D mesh input also includes its UV coordinates generated from the process in [35]. (Public domain Spot model by Keenan Crane)

ASOC source code. Therefore, for the sake of scope, PYAC does not implement this step as it is not directly included within the original ASOC codebase. Though, this UV unwrapping process is critical for generalizing the end-to-end occluding contour generation pipeline for any mesh other than the ones included with the ASOC researchers’ dataset. This UV unwrapping process is discussed as a potential addition to PYAC in Section 6.1.

3.2 Pipeline Overview

The following section provides an overview of the end-to-end pipeline from 3D mesh input to 2D occluding contour output as seen in Figure 3.2. Specifically, this section discusses the concepts seen in the ASOC paper and how they map to PYAC’s implementation. The theory behind this pipeline can be found in the original ASOC paper [11].

3.2.1 Mesh Input

The occluding contour generation pipeline starts by reading a 3D mesh Wavefront OBJ file – which contains its UV coordinates from the unwrapping process outlined in [35] – and utilizes such mesh for constructing an **affine manifold** and the edges between patch boundaries. For simplicity, the **affine manifold** allows for a representation of the mesh that is coordinate-agnostic and is subsequently independent of vertex coordinate positions. Additionally, the occluding contour generation

pipeline reads in a 4×4 extrinsic camera matrix, which contains the rotation r and translation t of the camera matrix M seen in Eq. (3.1). Within the PYAC codebase, the camera matrix is also referred to as the view matrix. With the camera matrix, an orthographic projection is applied to create the orthographic equivalent of the perspective projection.

$$M = \begin{bmatrix} r_{1,1} & r_{1,2} & r_{1,3} & t_x \\ r_{2,1} & r_{2,2} & r_{2,3} & t_y \\ r_{3,1} & r_{3,2} & r_{3,3} & t_z \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (3.1)$$

3.2.2 Surface Construction

The smooth surface represents the mesh as a set of patches $\mathbf{p}(u, v)$. It is worth noting that ASOC uses a modified version of geometry-based literature [46, 47] for this smooth surface construction, but its specific theory and workings are not covered in-depth for this paper. The smooth surface construction process is largely preserved in PYAC, though with adaptation to subclass parent constructor calls, member variable inheritance, and sparse matrix calculation. Here, PYAC utilizes *Cholespy* for its sparse matrix calculations, more of which is discussed later in Section 3.3.1.

3.2.3 Occluding Contour Calculation

After constructing the smooth surface and utilizing the UV unwrapping, PYAC can now utilize its smooth patches for calculating the occluding contours. The occluding contour calculation is broken into three substages: **generation**, **visibility calculation**, and **discretization** into either a rasterized image or SVG vector curves.

From the smooth surface, occluding contour **generation** can then occur on its patches, which is partially achieved by categorizing curves into conics such as ellipses, hyperbolas, intersecting lines, parabolas, or parallel lines. The importance of this categorization is discussed in ASOC’s paper, but will not be explored in this paper. Nonetheless, this conic categorization proves important for

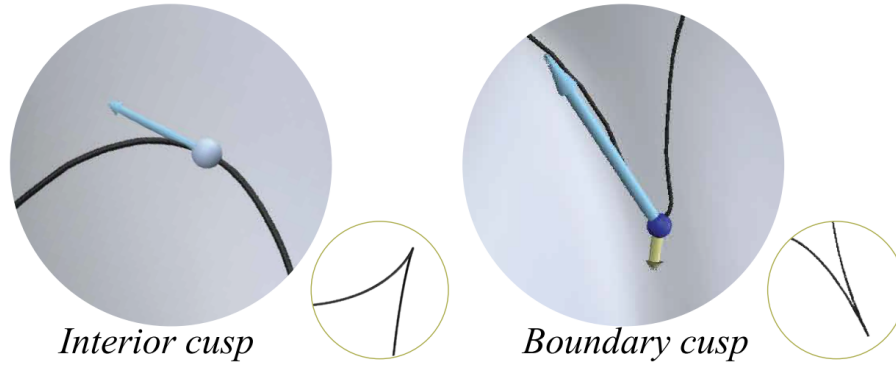


Figure 3.3: Interior cusps within surface patches and boundary cusps between patch boundaries (Figure from [11]).

PYAC’s implementation since it affects the evaluations later in Chapter 4. Within this **generation step**, the occluding contours go from their 3D space to 2D image-space curves. Additionally, the occluding contours are separated into **segments** based on their image-space intersections and their relationship to one another on the smooth surface representation. Furthermore, cases such as **cusps** prove important for visibility calculations since they denote where the algebraic curve changes direction (see Figure 3.3). These cusps are categorized into **interior cusps** and **boundary cusps**. Interior cusps denote cusps located within patches, while boundary cusps denote cusps at patch boundaries. These cusps affect the subsequent visibility calculation in that they denote where the occluding contour visibility may change. **Cones** are also important since occluding contours may share a point at these cone locations, which in turn affects the visibility calculation (see Figure 3.4). The specific reasoning for its importance is discussed in [11], but will not be covered further in this paper.

After generating the occluding contours, their **visibility** relative to the view direction is then calculated. ASOC utilizes a modified version of the Quantitative Invisibility algorithm [44] for this step, which they explain in their literature [11]. Concretely within the codebase, the Quantitative Invisibility allows users to input different modes of visibility calculation, those being direct, chaining, and propagation modes. The specific benefits of these modes are not relevant for the rest of the paper, but the **chaining** mode and the recursion-based **propagation** mode prove important later in Chapter 4.

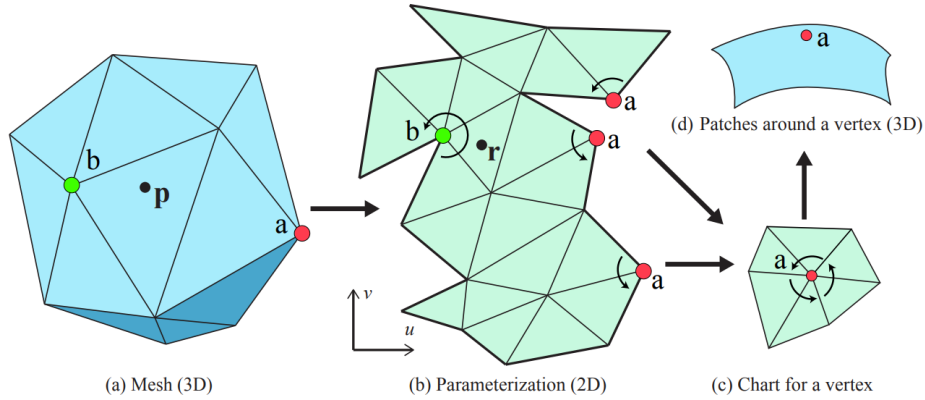


Figure 3.4: Cones denoted by the green vertices b . The specifics for cones are explored in [11] (Figure from [11]).

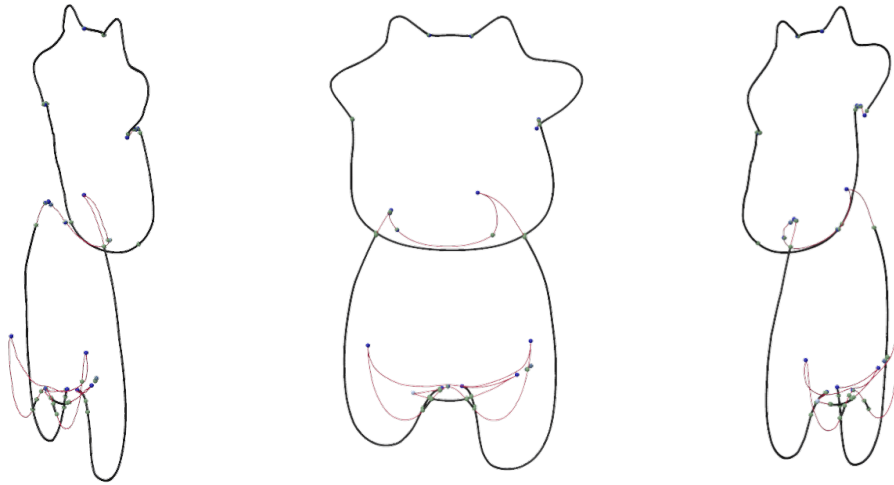


Figure 3.5: PYAC Polyscope wire screenshot in 3D space to showcase the line being single, closed form loop.

Lastly, in the pipeline, occluding contour **discretization** converts the occluding contours into either a **raster** or **vector** output. Note that for the *rasterized* output, PYAC currently utilizes Polyscope's screenshot functionality in correspondence with ASOC's implementation for such functionality.

3.3 Translation

This section focuses specifically on the faithful conversion from ASOC's C++ application to PYAC's Python library, which includes technical debt such as the preservation of implicit de-

dependencies, symbolic mathematics-generated functions, and direct floating point comparisons in ensuring correct functionality with ASOC and avoiding code failure. Unfortunately, this proves counter to PYAC’s goal of approachability since some of these translations prove difficult to debug, but overall highlights the need to improve upon these shortfalls via PYAC’s future developments.

3.3.1 Direct Dependencies

Aside from preserving file formats such as Wavefront OBJ for mesh reading, some ASOC libraries also have Python equivalents, with Table 3.1 highlighting these equivalent libraries. PYAC also preserves the module structure and file dependencies of ASOC.

C++ Package	Python Equivalent
CMake / CPM	pip
Eigen (MKL + BLAS + LAPACK)	NumPy
SuiteSparse	SciPy + Cholespy
CLII1	argparse
svg	svg.py
spdlog	logging
Polyscope	Polyscope
libigl	libigl
Doxygen	Sphinx

Table 3.1: ASOC C++ packages and PYAC Python equivalents.

Here, some translations directly mirror its C++ counterpart. So, *spdlog* and *CLII1* map to Python’s *logging* and *argparse* libraries, respectively. Other libraries such as *Polyscope* and *libigl* [48] for mesh visualization and parsing have Python bindings to utilize.

It is worth noting that various Cholesky solvers were explored such as those included with NumPy and SciPy for dense matrices. However, both were found inadequate for re-implementing ASOC since they provided too strict of mathematical guarantees, requiring that matrices are positive definite and throwing errors otherwise. Originally, Cholespy [49] also returned this error, but its codebase was modified to throw a warning instead, which matches the original behavior of ASOC. Also, Cholespy provides the benefit of sparse matrix factorization using CHOLMOD bindings, which provides improvements in performance. Though, it should be noted that this

modified Cholespy fork is not pip-installable online from PyPi, but is automatically included and installed from the external dependencies folder of PYAC’s source code. For both approachability and replicability, NumPy was chosen for its vectorized calculations, widespread support, various serialization methods, and also its user-friendly debugging features. This serves in contrast to ASOC’s Eigen, which does not natively support deserialization and whose availability cannot be guaranteed, as demonstrated by its Intel MKL dependency becoming unavailable in the Anaconda package repository and therefore requiring manual workarounds to resolve. SciPy saw utilization via its compressed sparse row matrix representation. For major functions in PYAC, Sphinx documentation strings replaced Doxygen for its popularity in various Python libraries and applications [50, 51]. This also allows for automated generation of documentation for PYAC, further aiding in approachability. Finally, svg.py was chosen for the occluding contour SVG output since it features a lightweight, Python-only library. Note that the popular alternative CairoSVG was initially considered but ultimately ignored since its functionality is reliant on the installation of Cairo onto the user’s system. This would hinder replicability for PYAC since Cairo is not natively included with operating systems such as Microsoft Windows.

3.3.2 Inkscape

Unfortunately, particular translations hinder approachability as their functionality largely remain undocumented and their style (i.e. variable and function naming) contrast with the rest of the codebase. Specifically, an implicit, hidden dependency between ASOC and Inkscape is featured within ASOC’s source code. Here, ASOC mentions an “Inkscope” in the context of computing occluding contour intersections. Though, upon closer inspection, “Inkscope” shares its code to the vector drawing tool Inkscape, particularly its descendant 2D geometry library lib2geom, also known as 2Geom [52]. It is possible that the ASOC developers did not require the entire functionality that lib2geom provided and opted for copying the functionality they needed. However, it is worth noting the style of this lib2geom code does not match the rest of ASOC and therefore proves counter to PYAC’s goal of approachability if translated directly. This lib2geom code was left as-is during

PYAC's implementation since changes to utilize NumPy's functionalities within this code resulted in failure in the overall end-to-end occluding contour pipeline.

3.3.3 Symbolic Mathematics

Symbolic mathematics applications (e.g. SymPy, MATLAB) were utilized in ASOC's development, at least from analysis of its source code. This sees heavy usage in the occluding contour calculation stage of the pipeline. These symbolic mathematics applications appear to have generated functions with optimizations that result in poor code readability, along with variable naming contributing to this issue. Optimizations such as recursion and utilizing specific C++ functionality provided a difficult implementation and direct translations of such symbolic mathematics-generated code remains unoptimal and counter to PYAC's approachability. It is also worth noting that in this symbolic mathematics-generated code, a floating point error that resulted from multiplication within the ray intersection calculation was noted and resulted in different logic branching paths between ASOC and PYAC and may prove important later in Chapter 4. With domain knowledge and comments within ASOC where the researchers provided the mathematics symbols used to generate certain code portions, one can recreate this code with a symbolic math library such as SymPy, as discussed in Section 6.1. For PYAC, these functions were translated as-is to preserve one of its goal of faithfulness to ASOC's original implementation. In addition, slight changes to the optimized auto-generated symbolic mathematics code resulted in code-breaking bugs, which further demonstrates the need for code translation faithfulness. However, as discussed in Section 6.1, utilizing symbolic math packages such as SymPy and examining the greater context for which these symbolic mathematics-generated code portions are utilized would prove helpful.

3.4 Localization

Some direct translations were not possible as a result of differences between ASOC's C++ and PYAC's Python such as class construction and variable initialization. Additionally, in some cases, arrays or vectors of simple data types were converted to NumPy instead of Python lists for consis-

tency with PYAC’s usage of NumPy. By leveraging NumPy, its functionalities such as preserving NaN values prove critical in maintaining numerical equivalence with ASOC’s calculations. Also, tolerances for some float equality statements have been adjusted to account for potential differences within PYACs calculations. Though, this may have implications for PYAC’s occluding contour output. Overall, ASOC originally used the Eigen library for its algebra and matrix computations, whose documentation proves extensive yet with significant differences between itself and NumPy. In the following sections, such differences fall in the line of matrix shapes and particular indexing functions.

3.4.1 Shape Checking

In the conversion from C++ to Python, many considerations were made to shape checking, which is for verifying matrix and array dimensionality. ASOC utilizes type checking to its advantage as it allows for clearer intent as to what size matrices are constructed or passed into functions. PYAC instead opts for various aliases and assertion statements in lieu of ASOC’s type checking functionality. Ultimately, this hindered approachability by creating code bloat. Section 6.1 proposes alternative approaches for addressing such issue.

3.4.2 Array Accessing

ASOC’s Eigen library and PYAC’s NumPy library differ through column and row accesses. In ASOC’s Eigen, 1D arrays are treated as 2D matrices. As a result, Eigen arrays with shape $(N, 1)$ or $(1, N)$ – where N represents the number of elements – are represented as $(N,)$ arrays in NumPy. In some cases, PYAC preserves the original dimensionality of ASOC’s Eigen $(N, 1)$ or $(1, N)$ matrices to support operations such as nested for loops. However, in other cases, these 2D matrices were simply flattened to shape $(N,)$ in correspondence with NumPy’s implementation of matrix calculations. Also, Eigen features row, column, and block accessing functions that PYAC adapted using NumPy’s bracket notation and slicing operators.

3.5 Test Suite

3.5.1 Pytest

PYAC features a Pytest suite for ensuring functional correctness with ASOC by utilizing ASOC's function input and output as the ground truth for correctness. Note that not all functions fared well with this one-to-one output comparison, particularly regarding contour intersections. This is likely the result of differing precision between ASOC and PYAC calculations and whose implications are explored in Chapter 4.

3.5.2 Serialization

To support the test suite, various **serialization** methods were added to more complex structures such as smooth patches, along with "setter" and "getter" functions added to classes to expose their parameters for debugging. Here, plain text formats such as JSON and CSV were utilized for data serialization and their simplicity while Wavefront OBJ files were used for consistency with ASOC. Though, this parameter exposure risks revealing unintentional access points for the user. This warrants consideration for encapsulation and other software engineering principles.

3.5.3 Branches

For the test suite, it should be noted that a separate branch of ASOC was made solely for serializing the output of its functions for the Pytest suite, which is completely separate from the ASOC branch used for evaluation in Chapter 4. In other words, there currently exists one ASOC branch for debugging function output for development and another branch as the control group for evaluating validity and performance in Chapter 4. Though, both branches have their CMake files corrected for Intel MKL dependency issues and utilized GCC 11.4.0 for compilation using the *release* build parameters and the ASOC-provided optimization flags.

3.6 Limitations

PYAC features various limitations resulting from inheriting technical debt from ASOC but also in its early design decisions that impact its future approachability. This includes various decisions made during the tradeoff between translation and localization of ASOC's code, dependencies within PYAC's code and test suite, along with edge cases that may result from PYAC's development.

3.6.1 Translation Versus Localization

While the balance between faithful translation versus functional localization was being worked on, core functionalities developed earlier in PYAC such as *rational functions* may still have C++-style variable naming (e.g. "m"-prefix private member variables). This process of balancing translation versus localization also extends to NumPy shape checking as well, with general components of PYAC such as *polynomials* opting for 2D arrays $(N, 1)$ rather than the 1D arrays $(N,)$ used often in the latter part of the occluding contour generation pipeline such as the *contour network* and *quadratic surface*. Additionally, other sections of PYAC's codebase, primarily the latter portion of its pipeline regarding the *smooth surface* and *contour generation*, still adhere to the C++ practice of modification-by-reference rather than Python's style of return-by-value. Such action was taken to mitigate bugs that would result from localization during development. Additionally, PYAC's accelerated development time meant that features such as documentation strings still remain under ASOC's Doxygen-based format. These discrepancies may impact PYAC's overarching goal of approachability as code style differences may result in confusion for developers working through the PYAC codebase.

3.6.2 Dependencies

Furthermore, the PYAC codebase features undocumented dependencies attributed to either its development process or technical debt inherited by ASOC's original C++ application. This includes

functionalities such as discretization or visualization being integral to the *occluding contour* or *smooth surface* classes rather than having their own separate classes. This also includes the subclassing of classes such as *conics* to *rational functions*, which may bring about a need for concern. Again, the implication for these dependencies are such that approachability may be impacted, as without documentation of these dependencies or mitigation to reduce such dependencies, the PYAC codebase builds technical debt that ends up making it unapproachable for its users and maintainers.

3.6.3 Testing

Comparing between ASOC and PYAC input and outputs proves cumbersome, requiring manual placement of serialization functions inside ASOC to then test. Additionally, function input and output serialized from ASOC were usually organized into their respective folders for de-serialization within PYAC via a manual process. Specifically, in the following directory structure:

```
[Source Mesh]/
├── [Camera File]/
│   └── [Function Name]/
```

If one were to expand the ground-truth test cases, this functionality should be made seamless. Though, as PYAC matures and its calculations are themselves mathematically sound, then the need to compare to the ground truth may provide lesser importance. It should be noted that in debugging the function output, improper versions of the Intel MKL dependencies were used. While this version issue was fixed for the evaluation in Chapter 4, this did not affect the design of the PYAC in drastic ways. Though, this potentially has implications for the test suite output, especially in cases where the outputs of ASOC and PYAC do not match identically as they may be utilizing invalid ASOC function outputs using the mismatching Intel MKL versions.

CHAPTER 4

EVALUATION

This section consists of two parts: **validity** and **performance**. These parts then have subsections evaluating **qualitative** and **quantitative** metrics. The validity evaluation includes *both* the *qualitative* visual similarity between the ASOC and PYAC smooth surfaces and occluding contours and also the *quantitative* data. This quantitative data includes metrics such as the number of vertices input, number of patches in the smooth surface, and also the number of segments and intersection in the occluding contour calculation. The rationale for including the quantitative data with the validity evaluation is for ensuring similar functional output between ASOC and PYAC. For example, if ASOC and PYAC both share the same number of segments and number of intersections alongside a final, nearly identical 2D occluding contour output, then it can be assumed that the functions calculating the occluding contours are valid. Note that the *number of intersections* is used interchangeably with the *number of intersection calls*, both referring to the number of intersection calculations performed.

The **performance** evaluation includes the **qualitative** visual similarity as part of its evaluation by writing vectorized and rasterized occluding contours to file. To ensure that both ASOC and PYAC performed equivalent computations to achieve the same occluding contour output, this file writing must occur. The **performance** evaluation also includes the **quantitative** runtime performance of ASOC and PYAC for the major sections of the occluding contour generation pipeline as outlined in Chapter 3.

4.1 Scope

For **scope**, particular metrics included in the original ASOC were irrelevant in both ASOC and PYAC. These include the number of internal and boundary cusps, and the number of ray intersection calls. While the number of internal cusps, boundary cusps, and ray intersection calls are

important for the Quantitative Invisibility calculation, their metrics were not updated in both ASOC and PYAC and are therefore omitted from the statistics table Table B.2. Additionally, the evaluation only examines the identity camera matrix at a Z-distance of 5. A smooth surface fitting weight of 1 was used throughout the evaluation, mirroring the decision of the original ASOC paper. Higher weights allow for greater "shrinkage" of the smooth surface to the underlying 3D surface (see Figure 3.1). Finally, the evaluation ran a total of 31 meshes for a singular run to calculate the figures and metrics for validity and performance.

4.2 Pipeline

The upcoming discussion of the pipeline entails the process for both **validity** and **performance** evaluations. Do note that the scripts perform the following list of actions, with particular emphasis on the Quantitative Invisibility method as it plays a role in the results later:

1. Randomly generates rotation matrix, which in this case is only the identity matrix.
2. Run ASOC **validity** evaluation executable with *chaining* Quantitative Invisibility, generating figures for visual comparison.
3. Run PYAC **validity** evaluation executable with *chaining* Quantitative Invisibility, generating figures for visual comparison.
4. Run ASOC **performance** evaluation executable with *propagation* Quantitative Invisibility, generating a CSV file for its results.
5. Run PYAC **performance** evaluation executable with *propagation* Quantitative Invisibility, generating a CSV file for its results.
6. Import both CSVs into Google Sheets for evaluating mean, maximum, minimum, and standard deviation metrics for **performance**
7. Visually inspect both ASOC and PYAC output figures for **validity** and quantitative results such as number of intersections and patches.

4.2.1 Modifications

To facilitate evaluation, the default ASOC repository did not support features that were desirable. Firstly, ASOC’s CMake files attempted to download Intel MKL from an invalid URL pointing to a now unavailable Anaconda package version. Therefore, a separate ASOC fork was made for installing these Intel MKL dependencies from those still available on the Anaconda package repository [53]. Secondly, scripts for generating validity and performance metrics were modified to support camera matrix input for equivalent view directions in both ASOC and PYAC. Additionally, the orthographic projection was applied on the input meshes before the smooth surface calculation, which originally did not occur. Note that Docker images are now available for archival purposes and were not used for the validity and performance evaluations.

The experiments were ran with the included experiments script in PYAC, which not only calls the PYAC evaluation scripts but also the modified ASOC executables built from the ASOC fork [53]. Here, PYAC adapted particular scripts for timing data and figure generation (i.e. occluding contours and smooth surface raster and vector images) from ASOC. These scripts were adapted from their C++ executables and Bash scripts in ASOC to Python scripts in PYAC for maximum compatibility across operating systems, consistent with PYAC’s overall goal of replicability.

4.2.2 Normalization

Additionally, in terms of the implementation of this evaluation, it is worth noting that the camera matrices included with the ASOC paper’s dataset also performed scaling on the input meshes as to fit them within the viewport, essentially performing vertex normalization. Therefore, if one were to utilize an extrinsic camera matrix only containing the rotation and translation of the camera, then the calculated occluding contours would appear outside of the visible viewport. Hence, both ASOC and PYAC were modified to perform vertex normalization on the input mesh rather than baking such operation into the camera matrix itself. The runtime for this normalization step in both ASOC and PYAC were measured in the evaluation as well.

In summary, the ASOC authors did not state that their dataset cameras also scaled the meshes

so that their vertices were normalized. Therefore, naively inputting a camera matrix that only performs rotation and translation would result in contours that were calculated outside the viewport.

4.2.3 Dataset

The original meshes that were tested upon were included as per the ASOC dataset. It should be noted that the models were already UV unwrapped using methods from [35] and whose UV unwrapping process is briefly discussed in Chapter 5.1 of [11]. In addition, the ASOC researchers performed pre-processing on their included dataset, which includes removal of particular mesh intersections and very short edges. This is important to note as it may have implications for replicability as the ASOC authors did not compare how the pre-processed meshes perform over the unprocessed meshes or if any errors occurred without any pre-processing.

Finally, as noted previously, an input camera matrix of the identity and a Z-distance of 5 was chosen, the camera matrix being the following format below where M represents the camera matrix:

$$M = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 5 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

4.2.4 Tooling

All experiments were run on a consumer desktop machine running Ubuntu 22.04 LTS with a Ryzen 9 5900 XT running at 3.30 GHz, 32 GB of DDR4 RAM running at 3200 MHz, and an Inland Professional 256 GB SSD over a SATA 3.0 port. Code from [11] was compiled on the same system using CMake 3.29.9 and GCC 11.4.0 with the *release* build, which removed assertions and optimized the program itself. Additionally, the Cholespy library was installed alongside PYAC's default installation command, which compiled the Cholespy library using its default *release* build settings. The dependencies for PYAC were installed via the included requirements file found in

the PYAC repository [39]. Additionally, since the modified version of Cholespy included with PYAC is not available in the PyPi package repository, it is included alongside PYAC’s source code and built locally, at least with the PYAC version as of the time of writing. The evaluations were executed from a Bash terminal via Microsoft’s Visual Studio Code, which was chosen for its simplicity. All other applications on the system were closed during the timing evaluations. For validity comparisons, the built-in photo viewer was used and both ASOC and PYAC figures were opened in their respective windows and manually compared.

4.3 Validity

For **validity**, scripts calling both ASOC and PYAC to generate figures for the affine manifold, smooth surface, and vector and raster occluding contours were made. These figures for ASOC and PYAC were then visually compared using the difference between pixel colors. Additionally, quantitative metrics including the number of intersection calls, number of faces, and number of patches were compared between ASOC and PYAC in ensuring that their pipelines generally performed the same number of calculations and thus ensuring functional validity. For the validity evaluation, the *chaining* Quantitative Invisibility mode is utilized for its overall stability. All figures are sorted in *ascending order* in terms of the number of faces in each mesh.

4.3.1 Quantitative Results

For mesh input, both ASOC and PYAC read the same **number of triangle faces**, which is expected considering both use libigl for mesh reading (see Table B.1 and Figure B.5). Additionally, both ASOC and PYAC generated the same **number of patches** (see Figure B.7) in their smooth surface calculation, which may demonstrate the robustness of the smooth surface construction to overall noise such as floating point errors. This stability is also reflected in PYAC’s development since the test suite cases for smooth surface construction utilizing ASOC’s function output as the ground truth generally succeeded.

However, where ASOC and PYAC differed is in their **number of segments** and **number of**

intersection calls. The number of segments were, on average, around 577 more than its ASOC counterpart (see Table B.1). Though, one would expect the same number of segments if PYAC had equivalent functional correctness with ASOC. The implications of this are discussed in Section 4.4.3. Additionally, ASOC utilizes a pass-by-reference variable for tracking the number of intersection calls. While an adaptation to PYAC was attempted, it appears non-functional as all the results for the number of intersections remain 0s. Overall, these results show the general instability of the occluding contour generation pipeline, of which care must be taken in ensuring functional correctness.

4.3.2 Qualitative Results

The generated figures for the affine manifold and the smooth surface both appear identical between ASOC and PYAC (see Table A.4 and Table A.5). This also holds true for the other meshes not visualized in these figures. Though, the pixel-wise difference figure shows entirely orange or partially orange figures (see Table A.1 for color mappings). This may simply be explained as a difference in shading rather than having any impact on the end-to-end occluding contour pipeline. Additionally, the ASOC paper also mentions that its figure feature different shading that does not impact its occluding contour calculations and is only used for visualization.

The generated **validity** figures for both *rasterized* and *vectorized* occluding contours appear similar, with rasterized occluding contours and the smooth surface appearing nearly identical between ASOC and PYAC aside from differences resulting from shading (see Table A.7 and Table A.8 for example). Specifically, the *raster* occluding contours utilizes the Polyscope viewer to capture its output, which inadvertently has the side effect of applying shading on the nodes (i.e. the dots seen in the images) and the occluding contours as well. This may explain why the pixel-wise difference figures in orange show that the occluding contours align pixel-wise, but do not correspond in color. Additionally, the nodes in the *vector* occluding contour figures are displayed in orange in the pixel-wise difference images (see Table A.7). This may be explained by the differences in the floating point representation of the RGB values for these nodes, which range from

0 to 1. Therefore, the ASOC and PYAC figures may have their nodes appear the same color to the human eye and on particular displays, but numerically are subtly different. This may also be a limitation with the color difference script used to generate these pixel-wise difference figures.

One small detail is that the "car" mesh's raster and vector occluding contours are missing a node in the top left corner of the full occluding contours ASOC output and its corresponding PYAC output (see the third and fourth column of Table A.2). This may simply be a difference in the numerical calculations between ASOC and PYAC as this does not appear to affect the visible occluding contours. Also, the "killeroo" mesh appears to have a smoother foot on the right side of its image (see right column of Table A.3) for only the ASOC outputs. This may demonstrate the inconsistency with ASOC's smooth surface-fitting implementation and may hint towards PYAC's robustness for replication.

4.4 Performance

For the **performance** evaluation, scripts calling both the modified ASOC fork and PYAC executables were made. The timing for the pipeline of both ASOC and PYAC were mostly measured within their codebase rather than the scripts themselves. Therefore, it is worth noting that the occluding contour generation process features various sections of code that measures their own runtime and also number of iterations called.

4.4.1 Recursion Failure

During the performance evaluation using the *propagation* Quantitative Invisibility method, the "car" mesh failed upon reaching the maximum recursion limit in PYAC. However, PYAC can generate the "car" mesh's occluding contours using the *chaining* Quantitative Invisibility. The reason for this being the case is that the "car" mesh appears to have the highest number of intersections out of all the meshes (see Figure B.6). This may reveal the upper limit of the recursive *propagation* method versus the *chaining* method of the Quantitative Invisibility.

4.4.2 Quantitative Results

For the performance quantitative results, the evaluations were separated by runtime and slowdown for the smooth surface construction and the occluding contour calculation pipeline. For the smooth surface construction, both ASOC and PYAC exhibit the same growth trend relative to the number of faces for each mesh (see Figure B.1). Though, on average, PYAC ran $20\times$ slower with around 19 seconds while ASOC ran around 1 second (see Table B.2). Also, as expected, an increase in the number of mesh faces appears to correspond to an increase in the runtime of the smooth surface construction.

For the occluding contour calculation, we can see that the trend of total occluding contour calculation time does not exhibit a clear linear trend with the number of faces (see Figure B.10 and Figure B.11). Rather, factors such as the number of intersections and also the number of segments may prove stronger predictors of occluding contour calculation runtime. Occluding contour calculation slowdown is visualized in Figure B.3 and its larger Figure B.2. Here, the occluding contour constructor call is in the order of magnitude of around $23,000\times$ slower than its ASOC counterpart. However, this is also considering that the ASOC mean runtime for the occluding contour construction call is 0.0009 seconds while PYAC performs 15.04 seconds on average. Relative to the overall occluding contour calculation pipeline, the occluding contour constructor call experiences the greatest slowdown (see Figure B.4). Overall, the PYAC total occluding contour calculation performs on average $480\times$ slower than its ASOC counterpart with an average runtime of around 32 seconds versus ASOC's 0.05 seconds (see Table B.2).

This discrepancy in PYAC's runtime may be explained by the high number of copies of complex data structures such as lists and NumPy arrays, which may lead to Python's garbage collector having to scan and clean these structures when they are no longer utilized. Subsequently, this then causes PYAC's implementation to slow down, especially at this occluding contour constructor call, which also performs calculations such as constructing the entire structure of occluding contours.

4.4.3 Qualitative Results

Additionally, figures were generated to ensure that the correct occluding contours were generated. Here, for particular meshes using *propagation* Quantitative Invisibility methods, ASOC **failed** in that some of its occluding contours for both vectorized and rasterized outputs had inconsistent visibility when compared to their *chained* equivalents during the **validity** calculation (see Table B.3 and Table B.4). This is despite the fact that they shared the same input matrix, same mesh data, and also called the same functions for writing vector and raster occluding contours to file.

Specifically, the "spot control", "ogre", "blub", "monsterfrog", "big guy", "pawn", "pig", "toad", and "torus" meshes all had occluding contour visibility in ASOC that were inconsistent with both PYAC and their chained counterparts. However, all the other meshes using *propagation* invisibility in the dataset fared just as well as their *chained* Quantitative Invisibility counterpart in the validity evaluation. Additionally, PYAC appears to have outputted visually identical occluding contours using the *propagation* method versus using the *chained* method. This may indicate the robustness of PYAC's implementation of both *chained* and *propagation* Quantitative Invisibility algorithms over ASOC's implementation.

The calculated number of segments for ASOC is also questionable since, as seen in Section 4.3.1, ASOC calculated noticeably less segments for its meshes overall. This may also impact the visual output of ASOC's occluding contours in the case of the *propagation* Quantitative Invisibility figures (e.g. "blub" and "big guy").

4.5 Overall

Overall, PYAC represents a base, unoptimized case of ASOC that amplifies which sections run the slowest, particularly the occluding contour calculation. Parts of the end-to-end pipeline such as the total occluding contour calculation runs on average $480\times$ slower than its ASOC equivalent with an average runtime of around 32 seconds compared to an average of 0.05 seconds in ASOC. However, PYAC's smooth surface construction runs around $20\times$ slower than its ASOC counterpart

with an average runtime of 18.76 seconds versus 0.9832 seconds in ASOC. Both of these results may indicate performance issues with PYAC, especially in regards to its garbage collector.

However, in terms of validity, PYAC has shown to be equivalent to that of ASOC’s output. In some cases, PYAC demonstrates its robustness through its consistent occluding contours output across the chaining and propagation Quantitative Invisibility methods. Additionally, PYAC’s evaluation has shown that ASOC’s recursive propagation visibility calculations reach their depth limit in PYAC.

4.5.1 Limitations

It would have been more accurate to sort the meshes by their number of intersection calls rather than their number of faces for occluding contour-related calculations since the former appears to be a stronger indicator of runtime than the latter. However, for the smooth surface construction, the number of faces is a stronger indicator than the number of intersection calls.

Also, note that the evaluation was ran for a total of one trial of 31 meshes. A more accurate evaluation would utilize a larger number of trial runs and would coalesce their results. However, the evaluation between ASOC and PYAC for both validity and performance ran around roughly 1 hour, which made such evaluations difficult to achieve. Nonetheless, the evaluations indicated parts of PYAC’s code that may need optimization and particular cases where PYAC robustness is greater than that of ASOC.

Additionally, the qualitative analysis is currently based on a visual comparison between the ASOC and PYAC output and would benefit from automated processes, especially since features like its nodes are difficult to visually inspect as they are densely packed in the figures (e.g. the ”car” mesh in Table A.2). Therefore, automated processes to ensure validity between ASOC and PYAC outputs would also help in scaling the evaluation between ASOC and PYAC for a larger number of trial runs.

Furthermore, the experiments are limited to the meshes included with the ASOC dataset. Though, this is a result of ASOC’s reliance on an external UV unwrapping application to gen-

erate its meshes' UV coordinates. Additionally, not all camera view directions were tested as only the identity matrix was used. Alternative camera view directions may correspond to varying number of segments and intersection calls, which would help provide a more diverse set of examples for evaluation.

4.5.2 Dependent and Independent Views

The evaluation can be broken down into two sections: **view independent** operations and **view dependent** operations. Simply put, view independent operations do not require the projected vertex data to operate, only relying on information such as the faces and the UV unwrapping parametrization while view *dependent* operations require such vertices depending on the view direction. However, this distinction does not need an explanation in this write up because it sees usage within the code itself. Specifically, ASOC and PYAC view dependent operations involve the smooth surface and occluding contour calculation themselves. Meanwhile, view-independent operations such as the affine manifold calculation do not require updated vertices depending on view direction. Instead, it requires non-view dependent information such as faces, UV coordinates, and face textures. This distinction is not relevant for the paper, but is worth noting for context.

4.5.3 Assertion False Positives

It is also worth noting that the *debug* build of ASOC fails somewhere along affine manifold's usage in its pipeline as various false positive assertions are triggered, causing the pipeline to fail prematurely. Additionally, sections of the conic parameterization (i.e. the occluding contour generation categorizes occluding contours to hyperbolas, parallel lines, intersection lines, etc.) also triggers false positives in both the ASOC and PYAC code, causing premature failure. This is not of major concern in the evaluation results as assertion and logging statements are deactivated in both ASOC and PYAC, but is worth noting for anyone redoing the evaluation experiments.

CHAPTER 5

DISCUSSION

Chapter 5 focuses on related literature to PYAC and helps provide an intuition around the greater landscape of NPR line art research. This discussion is not limited to occluding contours and also focuses on alternative approaches to line art generation.

5.1 Alternative Works

Despite not serving as the focus of this paper, it is worth mentioning that **alternative geometric representations** most recently include Meschers, which aims at answering the problem of rendering impossible objects in 3D and whose authors note ASOC as a potential research direction [54]. Work has also explored the **psychological** connection to line art and the mapping of hand-drawn strokes to categories such as apparent ridges and occluding contours [55, 56, 57, 58].

5.2 Tooling

Freestyle provides programmable line art qualities such as stroke style using Python [10]. Freestyle also supports various line types such as occluding contours and outlines via its module-based approach. The Freestyle paper also sees its implementation into Blender as well [26]. However, Blender features a newer tool for line art generation called Line Art, which utilizes Blender's Grease Pencil objects rather than an independent render engine like Blender's Freestyle. Note that Grease Pencil is one of the ways that Blender allows its users to draw pencil strokes in 3D.

5.3 Validity Theory

Alternatively, ConTesse proposes a mesh-based approach addressing the line art stability problem by generating a new 3D mesh that is essentially an optimization of subsurface divisions around

occluding contours [14]. ConTesse and the grander curve validity theory both formalize the line art stability problem, stating that curves will face instability unless its surface can facilitate such curve generation [43].

5.4 GPU Approaches

Other line art approaches have opted for parallelization, harnessing the power of the GPU in overcoming the issue of lengthy computation times and providing real-time occluding contour generation. [12] approaches occluding contour generation by utilizing compute shaders for occluding contour visibility and chaining, which connects the occluding contours. Its subsequent Blender fork *StrokeGen* showcases this novel approach to occluding contour generation [59]. [60] builds upon this work utilizing alternative data structures and algorithms.

5.5 Machine Learning

Rapid advancements in machine learning within the past decade has enabled its integration into many computer science disciplines, with computer graphics being no exception. One such integration of machine learning takes an alternative approach from ASOC in that it uses machine learning in examining particular regions of an image to simplify generated line art from a 3D mesh [13]. This operates in the image space and can potentially be used in improving visual clarity in line art drawings. For stylization, it aims for simplicity rather than complexity, removing unnecessary lines that do not contribute to the overall design language.

Alternatively, other machine learning approaches opt for operating entirely in 3D such as WIR3D [37]. Rather than following the traditional 3D to 2D line art route, WIR3D proposes a method that optimizes 3D cubic Bézier curves in 3D space. So, rather than the end product being contours in a 2D image plane, WIR3D creates strokes in a 3D space as its end product. It is also worth noting that WIR3D uses Blender Freestyle in its experimental design.

CHAPTER 6

CONCLUSION

Line art plays a pivotal role in NPR research and especially in creative works. Here, NPR line art sees various applications in games and animation where line art has pushed the envelope for creative expression. However, such techniques rely on previous approaches that either provided unstable line art or were computationally expensive. ASOC addresses this stability issue via a smooth surface construction of the mesh for occluding contour generation, but encounters issues in terms of replicability and approachability. Therefore, PYAC improves upon ASOC’s limitations in replicability and approachability through its documentation and Python-based implementation permitting greater access among users such as researchers and technical artists. In my evaluation, PYAC’s failure in particular cases resulted from its re-implementation of ASOC’s functionality. Particular re-implementations faced significant slowdowns such as PYAC’s occluding contour, which experienced a $480\times$ slowdown with an average runtime of 32 seconds relative to ASOC’s 1 second runtime in a limited set meshes and a singular view direction. However, the same evaluation also found cases demonstrating ASOC’s lack of robustness and inconsistency in its generated line art.

6.1 Future Work

The upcoming section focuses on bug fixes in PYAC’s source code and also longer-term research directions using PYAC. Though, any future work should first evaluate if the UV unwrapping step in [35] is generalizable across different manifold meshes since the limitations with the UV unwrapping process may also extend to PYAC and hinder any further progress.

6.1.1 Urgent Fixes

The following includes immediate work that should be conducted on PYAC before proceeding with the other future work. This is important to reduce technical debt that may accumulate from PYAC's early development.

ASOC's fragile operations hindered progress in localization and approachability that eventually propagated to PYAC. Operations that rely on symbolic mathematics libraries or applications (e.g. SymPy and MATLAB) should instead call these libraries' functions directly, rather than using their auto-generated code. There should also be a shift towards Python's style of operations. This includes variable naming, Sphinx documentation, and type-checking. For type-checking, tools such as *Mypy* may prove useful over PYAC's current implementation of aliases and shape assertions. Future work should provide clearer documentation of dependencies and also establish a proper public-facing API. This would help distinguish which aspects of PYAC are meant for maintainers versus researchers, technical artists, and other users. After satisfying these objectives, it will then be safe for applications to be built using PYAC.

6.1.2 Longer-term Research

Testing with additional meshes and creating their UV unwrappings would help evaluate PYAC's robustness. This is particularly relevant since the UV unwrapping code from [35] provides a Python script to aid in the generation of these UV coordinates. Future work can also document PYAC's coordinate system since such information is required when integrating PYAC into applications such as a Blender add-on. A formal evaluation of PYAC compared to line art tools such as Blender's Line Art and also Pencil+4 may provide an interesting research direction. This research direction could then analyze the features that these tools offer and how PYAC can be integrated.

REFERENCES

- [1] M. C. Stone and M Stone, “A survey of color for computer graphics,” *Course at SIGGRAPH*, vol. 744, 2001.
- [2] H. Qin, *Non-Photorealistic Rendering*, <https://www3.cs.stonybrook.edu/~qin/courses/graphics/graphics-non-photorealistic-rendering.pdf>, Online Slides; accessed 23-February-2026, Stony Brook University, 2021.
- [3] B. Gooch and A. Gooch, *Non-photorealistic rendering*. AK Peters/CRC Press, 2001.
- [4] T. Strothotte and S. Schlechtweg, *Non-photorealistic computer graphics: modeling, rendering, and animation*. Morgan Kaufmann, 2002.
- [5] A. Gooch, B. Gooch, P. Shirley, and E. Cohen, “A non-photorealistic lighting model for automatic technical illustration,” in *Proceedings of the 25th annual conference on Computer graphics and interactive techniques*, 1998, pp. 447–452.
- [6] A. Lake, C. Marshall, M. Harris, and M. Blackstein, “Stylized rendering techniques for scalable real-time 3d animation,” in *Proceedings of the 1st international symposium on Non-photorealistic animation and rendering*, 2000, pp. 13–20.
- [7] S. Rusinkiewicz, M. Burns, and D. DeCarlo, “Exaggerated shading for depicting shape and detail,” *ACM Transactions on Graphics (TOG)*, vol. 25, no. 3, pp. 1199–1205, 2006.
- [8] P. B  nard, A. Hertzmann, and M. Kass, “Computing smooth surface contours with accurate topology,” *ACM Transactions on Graphics (TOG)*, vol. 33, no. 2, pp. 1–21, 2014.
- [9] S. Grabli, E. Turquin, F. Durand, and F. Sillion, “Programmable style for npr line drawing,” in *Rendering Techniques 2004 (Eurographics Symposium on Rendering)*, ACM Press, 2004.
- [10] S. Grabli, E. Turquin, F. Durand, and F. X. Sillion, “Programmable rendering of line drawing from 3d scenes,” *ACM Transactions on Graphics (TOG)*, vol. 29, no. 2, pp. 1–20, 2010.
- [11] R. Capouellez, J. Dai, A. Hertzmann, and D. Zorin, “Algebraic smooth occluding contours,” in *ACM SIGGRAPH 2023 Conference Proceedings*, ser. SIGGRAPH ’23, Los Angeles, CA, USA: Association for Computing Machinery, 2023, ISBN: 9798400701597.
- [12] W. Jiang, G. Li, Y. Nie, and C. Xian, “Gpu-driven real-time mesh contour vectorization,” in *EGSR (ST)*, 2022, pp. 93–105.

- [13] V. Nguyen, M. Fisher, A. Hertzmann, and S. Rusinkiewicz, “Region-aware simplification and stylization of 3d line drawings,” in *Computer Graphics Forum*, Wiley Online Library, vol. 43, 2024, e15042.
- [14] C. Liu, P. Bénard, A. Hertzmann, and S. Hoshyari, “Contesse: Accurate occluding contours for subdivision surfaces,” *ACM Trans. Graph.*, vol. 42, no. 1, Feb. 2023.
- [15] A. Hertzmann and D. Zorin, “Illustrating smooth surfaces,” in *Proceedings of the 27th annual conference on Computer graphics and interactive techniques*, 2000, pp. 517–526.
- [16] D. Decarlo, A. Finkelstein, S. Rusinkiewicz, and A. Santella, “Suggestive contours for conveying shape,” *ACM Transactions on Graphics*, vol. 22, no. 3, pp. 848–855, 2003.
- [17] Y. Ohtake, A. Belyaev, and H.-P. Seidel, “Ridge-valley lines on meshes via implicit surface fitting,” in *ACM SIGGRAPH 2004 Papers*, 2004, pp. 609–612.
- [18] T. Judd, F. Durand, and E. Adelson, “Apparent ridges for line drawing,” *ACM transactions on graphics (TOG)*, vol. 26, no. 3, 19–es, 2007.
- [19] D. DeCarlo and S. Rusinkiewicz, “Highlight lines for conveying shape,” in *Proceedings of the 5th International Symposium on Non-photorealistic Animation and Rendering*, 2007, pp. 63–70.
- [20] K. Karsch and J. C. Hart, “Snaxels on a plane,” in *Proceedings of the ACM SIGGRAPH / Eurographics Symposium on Non-Photorealistic Animation and Rendering*, 2011, pp. 35–42.
- [21] R. Sayeed and T. Howard, “State of the art non-photorealistic rendering (NPR) techniques,” in *TPCG*, 2006, pp. 89–98.
- [22] A. Mohr, E. Bakke, A. Gardner, C. Herrman, and S. Dutcher, *NPRQuake*, <https://research.cs.wisc.edu/graphics/Gallery/NPRQuake/>, Online; accessed 24-February-2026, 2004.
- [23] J. Mitchell, *Stylization with a Purpose*, https://cdn.fastly.steamstatic.com/apps/valve/2008/GDC2008_StylizationWithAPurpose_TF2.pdf, Online Slides; accessed 5-March-2026, Valve Corporation, 2008.
- [24] L. Pope, *Return of the Obra Dinn*, <https://dukope.com/devlogs/obra-dinn/tig-00/>, Online Blog; accessed 3-March-2026, 2014.
- [25] L. Pope, *Return of the Obra Dinn*, <https://forums.tigsource.com/index.php?topic=40832>, Online Blog; accessed 3-March-2026, 2014.

- [26] M. Curioni and T. Kajiyama, *Freestyle integration into Blender*, <https://freestyleintegration.wordpress.com/>, Online Blog; accessed 9-February-2026, 2017.
- [27] Y. Wu, *Proposal: Blender NPR line rendering improvement*, https://archive.blender.org/wiki/2024/wiki/User:Yiming/Proposal_2018.html, Online; accessed 6-March-2026, 2018.
- [28] Y. Wu, *LANPR 2019 proposal*, <https://archive.blender.org/wiki/2024/wiki/User:Yiming/GSoC2019/Proposal.html>, Online; accessed 6-March-2026, 2019.
- [29] Y. Wu, *Line art*, <https://www.wellobserve.com/en.php?post=20211021001236>, Online; accessed 6-March-2026, 2025.
- [30] [n.a.], <https://malt3d.com/>, Online; accessed 3-March-2026, [n.y.]
- [31] F. Clément, *NPR project*, <https://code.blender.org/2025/05/npr-project/>, Online Forum; accessed 22-February-2026, 2025.
- [32] F. Clément, *NPR project & multi-stage compositing*, <https://devtalk.blender.org/t/npr-project-multi-stage-compositing/40758>, Online Forum; accessed 22-February-2026, 2025.
- [33] N. Bonneel, D. Coeurjolly, J. Digne, and N. Mellado, “Code replicability in computer graphics,” *ACM Transactions on Graphics (TOG)*, vol. 39, no. 4, pp. 93–1, 2020.
- [34] D. Panozzo, *Graphics replicability stamp initiative*, <https://www.replicabilitystamp.org/>, Online; accessed 4-February-2026, [n.y.]
- [35] M. Campen, R. Capouellez, H. Shen, L. Zhu, D. Panozzo, and D. Zorin, “Efficient and robust discrete conformal equivalence with boundary,” *ACM Transactions on Graphics (TOG)*, vol. 40, no. 6, pp. 1–16, 2021.
- [36] L. Huang and G. Levin, *ApparentRidges*, <https://github.com/CreativeInquiry/ApparentRidges>, Github Repository; accessed 24-February-2026, 2020.
- [37] R. Liu, D. Fu, N. Tan, I. Lang, and R. Hanocka, “Wir3d: Visually-informed and geometry-aware 3d shape abstraction,” *arXiv preprint arXiv:2505.04813*, 2025.
- [38] K. Ha, *Ranato*, <https://github.com/mk39-png/Ranato>, Github Repository; accessed 5-January-2026, 2026.
- [39] K. Ha, *PYAC: Python algebraic contours*, <https://github.com/mk39-png/py-algebraic-contours>, Github Repository; accessed 20-March-2026, 2026.

- [40] A. Vilanova, *Non-photorealistic rendering*, <https://www.cg.tuwien.ac.at/courses/CG2/SS2002/NPR.pdf>, Online; accessed 4-March-2026, Vienna University of Technology, 2023.
- [41] S. Rusinkiewicz, F. Cole, D. DeCarlo, and A. Finkelstein, “Line drawings from 3d models,” in *ACM SIGGRAPH 2008 classes*, 2008, pp. 1–356.
- [42] P. B  nard, A. Hertzmann, et al., “Line drawings from 3d models: A tutorial,” *Foundations and Trends   in Computer Graphics and Vision*, vol. 11, no. 1-2, pp. 1–159, 2019.
- [43] A. Hertzmann, “New insights in smooth occluding contours for nonphotorealistic rendering,” *IEEE Computer Graphics and Applications*, vol. 44, no. 1, pp. 76–85, 2024.
- [44] A. Appel, “The notion of quantitative invisibility and the machine rendering of solids,” in *Proceedings of the 1967 22nd national conference*, 1967, pp. 387–393.
- [45] A. Hertzmann, *Occluding contour breakthroughs, part 1: A surprisingly hard problem*, <https://aaronhertzmann.com/2023/07/31/occluding-contours-part-1.html>, Online Blog; accessed 19-March-2026, 2023.
- [46] W. Dahmen, “Smooth piecewise quadric surfaces,” in *Mathematical methods in computer aided geometric design*, Elsevier, 1989, pp. 181–193.
- [47] Y. He, M. Jin, X. Gu, and H. Qin, “Ac 1 globally interpolatory spline of arbitrary topology,” in *International Workshop on Variational, Geometric, and Level Set Methods in Computer Vision*, Springer, 2005, pp. 295–306.
- [48] A. Jacobson and D. Panozzo, “Libigl: Prototyping geometry processing research in c++,” in *SIGGRAPH Asia 2017 courses*, 2017, pp. 1–172.
- [49] J. Schmidt, W. Jakob, and B. Nicolet, *Cholespy: An easily integrable Cholesky solver on CPU and GPU*, <https://github.com/rgl-epfl/cholespy>, Github Repository; accessed 15-December-2025, [n.y.]
- [50] [n.a.], *Blender Python API documentation*, <https://docs.blender.org/api/current/index.html>, Online Documentation; accessed 22-December-2025, [n.y.]
- [51] A. Sampson, *Beets documentation*, <https://beets.readthedocs.io/en/stable/plugins/index.html>, Online Documentation; accessed 12-February-2026, 2016.
- [52] [n.a.], *Inkscape code*, https://inkscape.gitlab.io/inkscape/doxygen/bezier-clipping_8cpp_source.html, Online Documentation; accessed 12-February-2026, [n.y.]

- [53] R. Capouellez, J. Dai, A. Hertzmann, and D. Zorin, *Algebraic smooth occluding contours Github repository*, <https://github.com/mk39-png/algebraic-contours>, Fork of Algebraic Smooth Occluding Contours Github Repository; accessed 21-March-2026, [n.y.]
- [54] A. Dodik et al., “Meschers: Geometry processing of impossible objects,” *ACM Transactions on Graphics (TOG)*, vol. 44, no. 4, pp. 1–10, 2025.
- [55] F. Cole et al., “Where do people draw lines?” In *ACM SIGGRAPH 2008 papers*, 2008, pp. 1–11.
- [56] F. Cole et al., “How well do line drawings depict shape?” In *ACM SIGGRAPH 2009 papers*, 2009, pp. 1–9.
- [57] H. Sheng, J. Wilder, and D. B. Walther, “Where to draw the line?” *PLoS One*, vol. 16, no. 11, e0258376, 2021.
- [58] A. Hertzmann, “The role of edges in line drawing perception,” *Perception*, vol. 50, no. 3, pp. 266–275, 2021.
- [59] W. Jiang, *Strokegenreleases*, <https://github.com/JiangWZW/strokegen-releases>, Github Repository; accessed 20-March-2026, 2024.
- [60] P. Tsiapkolis and P. Bénard, “Patch decomposition for efficient mesh contours extraction,” in *Computer Graphics Forum*, Wiley Online Library, vol. 43, 2024, e15154.

Appendices

APPENDIX A

QUALITATIVE EVALUATION

Color	Explanation
Red	Pixel present only in ASOC
Blue	Pixel present only in PYAC
Purple	Same pixel position, same color (i.e. identical pixel)
Orange	Same pixel position, different color
White	Denotes the background color

Table A.1: Explanation of colors in pixel-wise difference figures.

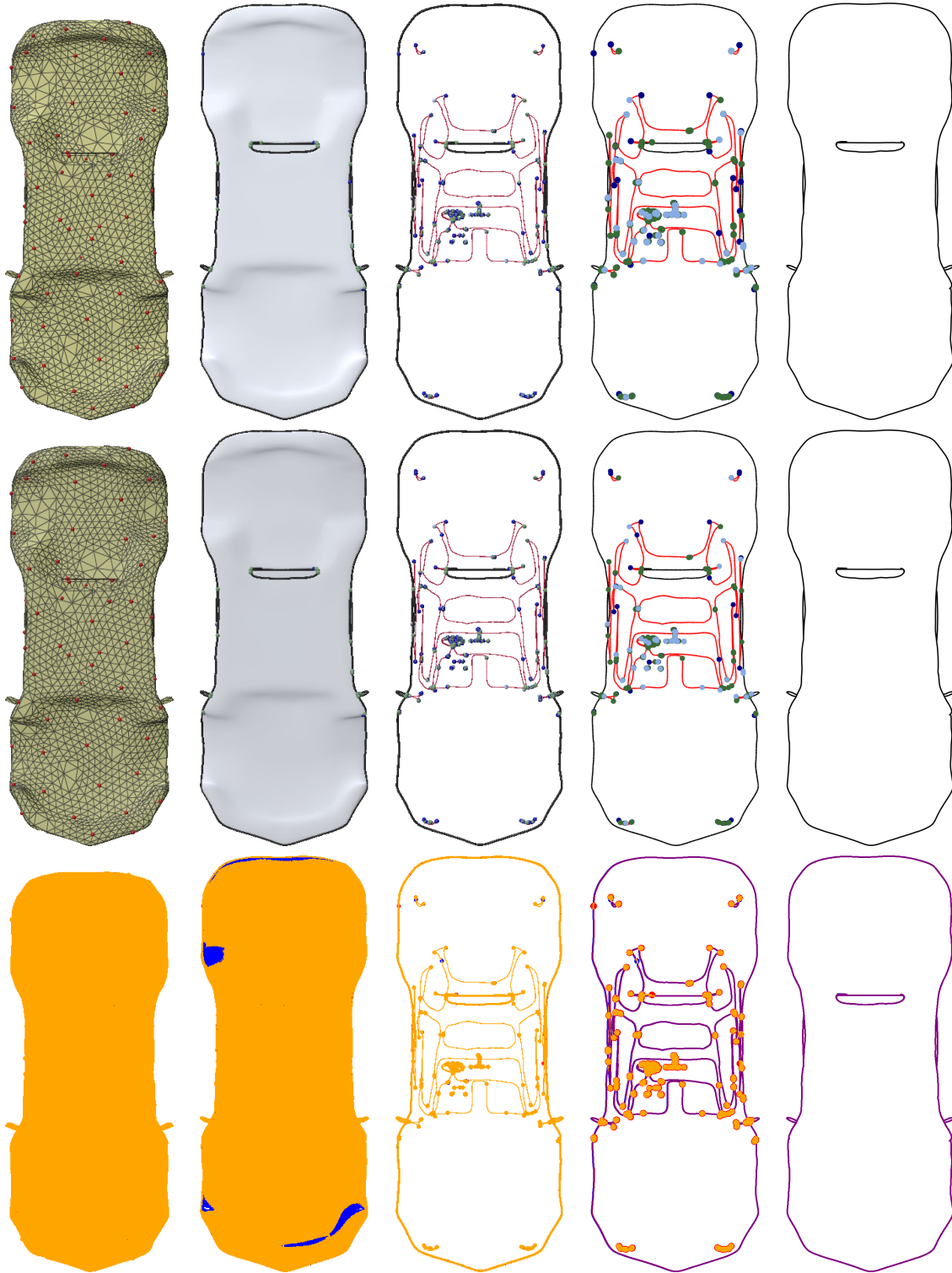


Table A.2: Car affine manifold, algebraic smooth surface, raster full occluding contours, vector full occluding contours, and vector visible occluding contours using chaining Quantitative Invisibility (left to right) (ASOC (top) vs PYAC (middle) vs difference (bottom)) (see Table A.1 for color map) (Car ©Pixar).

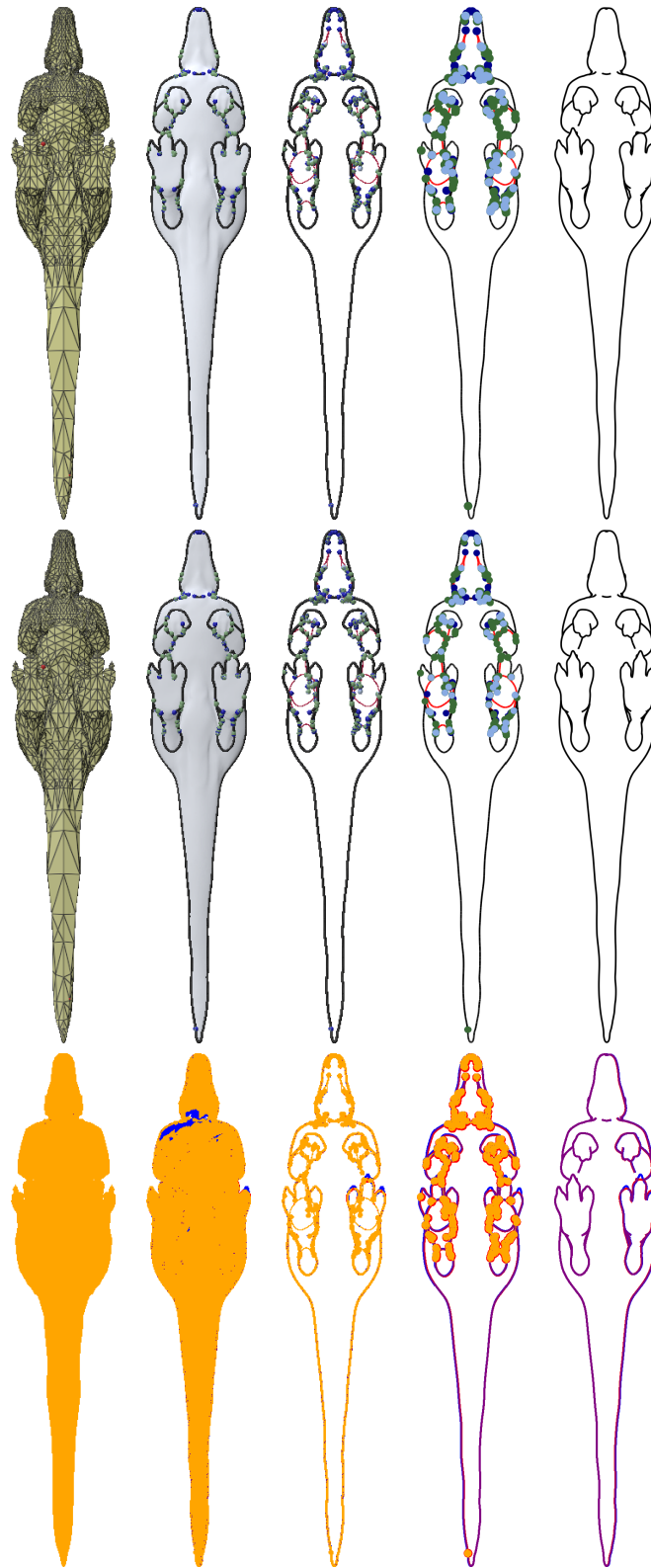


Table A.3: Killeroo affine manifold, algebraic smooth surface, raster full occluding contours, vector full occluding contours, and vector visible occluding contours using chaining Quantitative Invisibility (left to right) (ASOC (top) vs PYAC (middle) vs difference (bottom)) (see Table A.1 for color map) (Killeroo ©headus.com.au.).

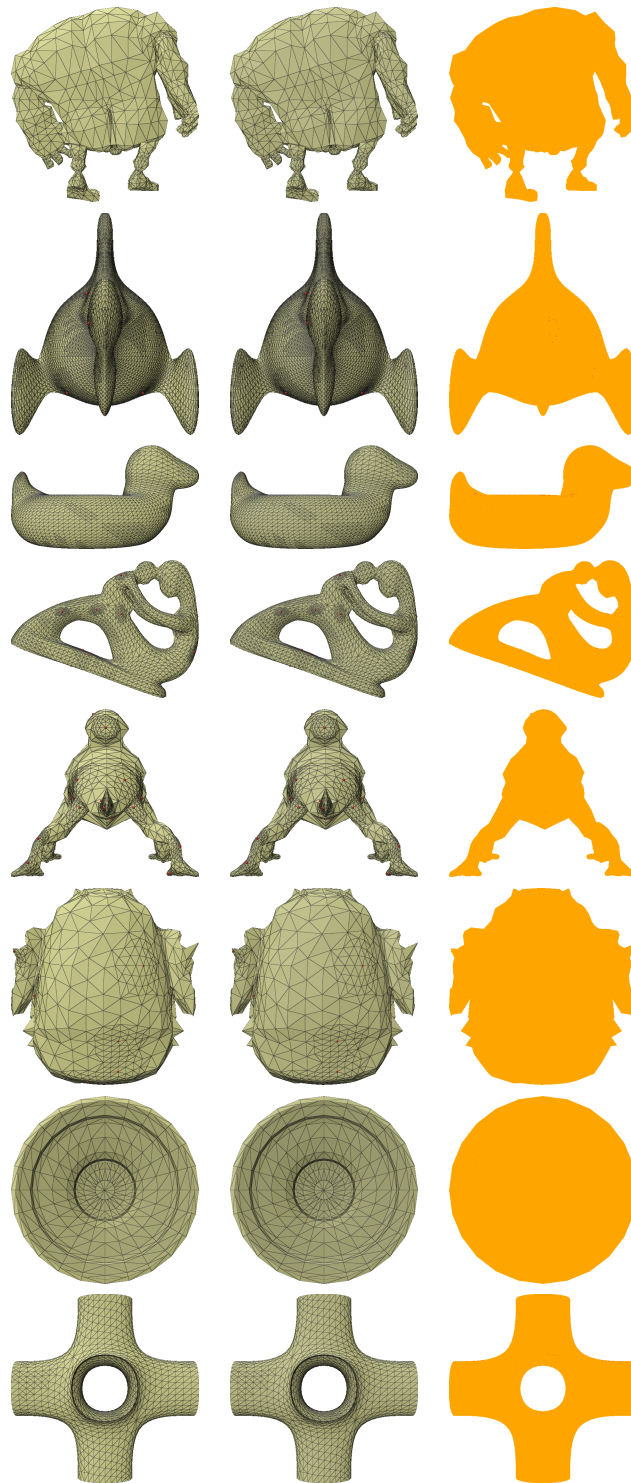


Table A.4: Raster affine manifold using chaining Quantitative Invisibility (ASOC (left) vs PYAC (middle) vs difference (right)) (see Table A.1 for color map) (Bigguy and Monster Frog ©Bay Raitt. Fertility from AIM@SHAPE-VISIONAIR Shape Repository. Public domain Blub, Bob, and Ogre models by Keenan Crane. Open source Pawn model from OpenSubdiv.).

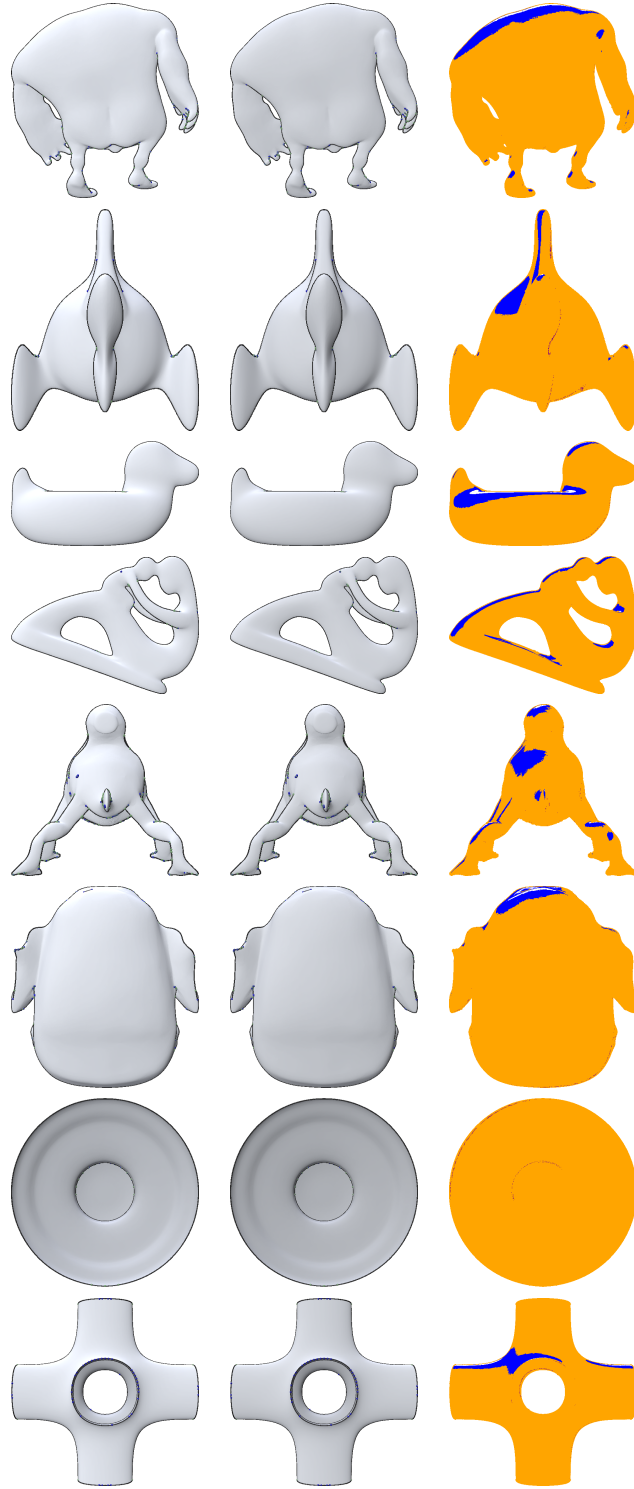


Table A.5: Raster smooth surface and visible occluding contours using chaining Quantitative Invisibility (ASOC (left) vs PYAC (middle) vs difference (right)) (see Table A.1 for color map) (Bigguy and Monster Frog ©Bay Raitt. Fertility from AIM@SHAPE-VISIONAIR Shape Repository. Public domain Blub, Bob, and Ogre models by Keenan Crane. Open source Pawn model from OpenSubdiv.).

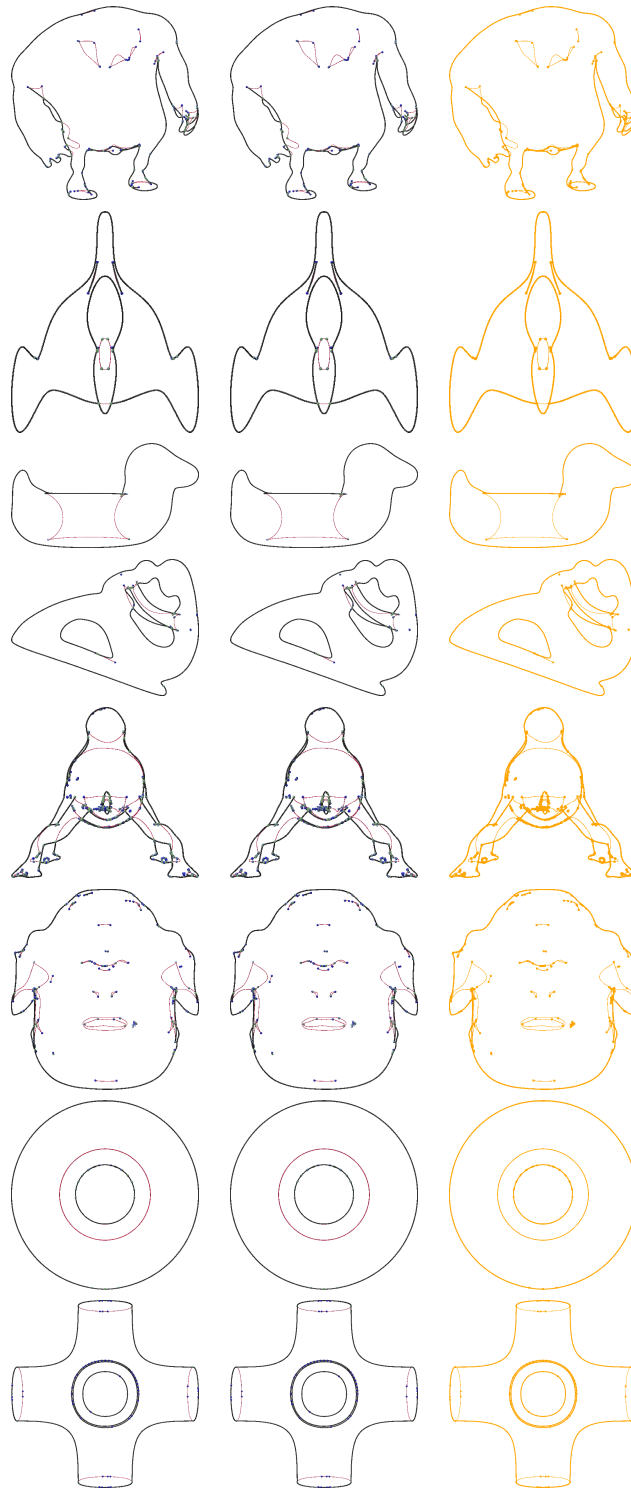


Table A.6: Raster full occluding contours using chaining Quantitative Invisibility (ASOC (left) vs PYAC (middle) vs difference (right)) (see Table A.1 for color map) (Bigguy and Monster Frog ©Bay Raitt. Fertility from AIM@SHAPE-VISIONAIR Shape Repository. Public domain Blub, Bob, and Ogre models by Keenan Crane. Open source Pawn model from OpenSubdiv.).

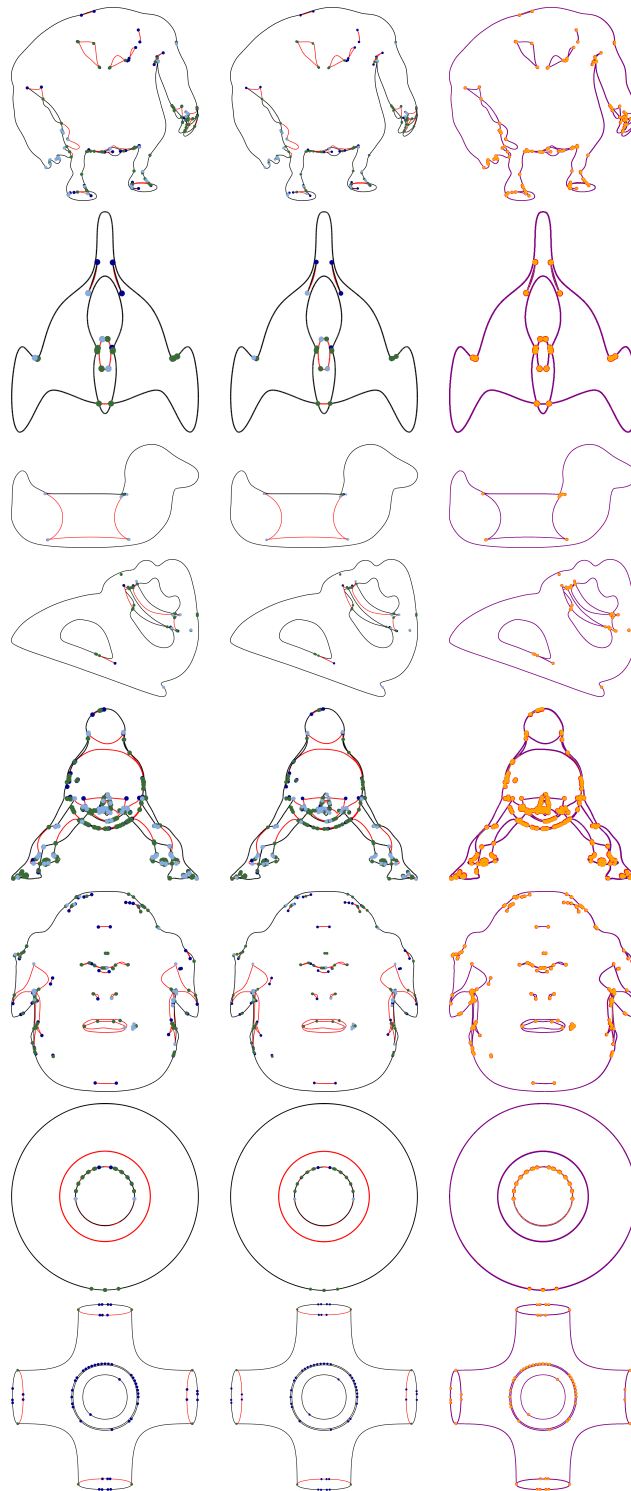


Table A.7: Vector full occluding contours using chaining Quantitative Invisibility (ASOC (left) vs PYAC (middle) vs difference (right)) (see Table A.1 for color map) (Bigguy and Monster Frog ©Bay Raitt. Fertility from AIM@SHAPE-VISIONAIR Shape Repository. Public domain Blub, Bob, and Ogre models by Keenan Crane. Open source Pawn model from OpenSubdiv.).

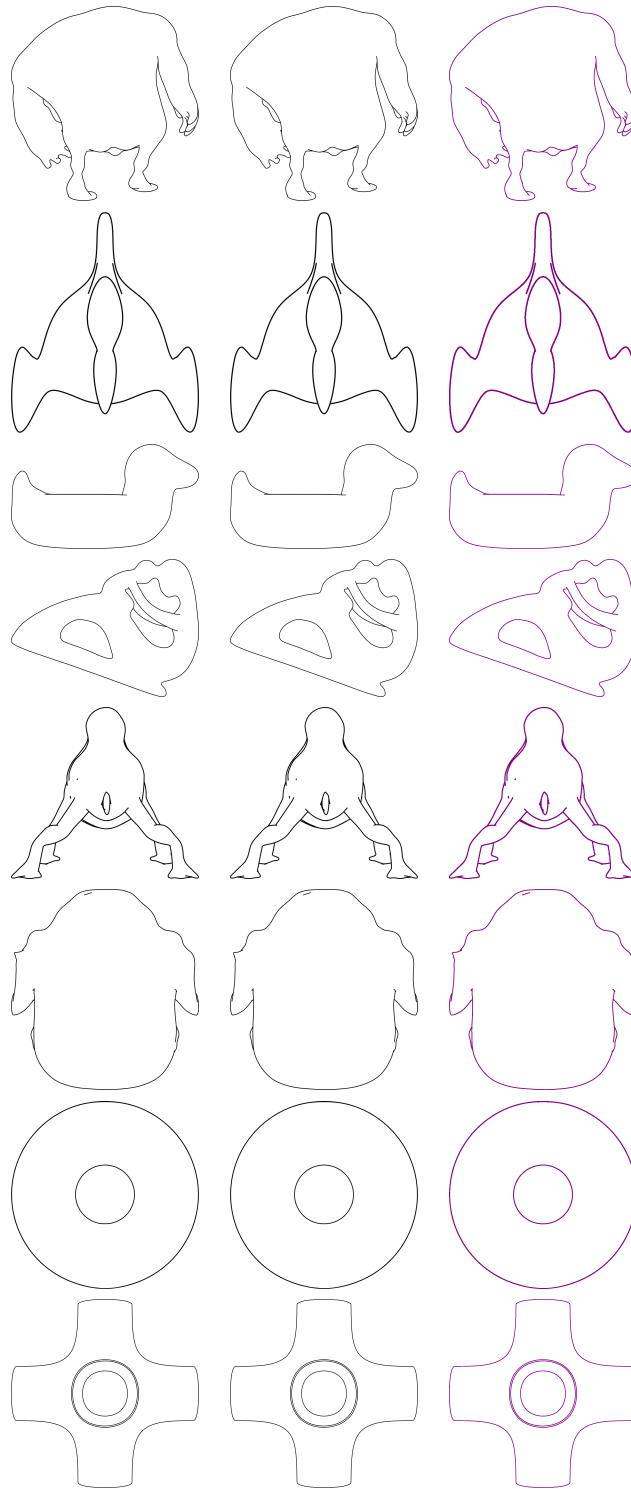


Table A.8: Vector visible occluding contours using chaining Quantitative Invisibility (ASOC (left) vs PYAC (middle) vs difference (right)) (see Table A.1 for color map) (Bigguy and Monster Frog ©Bay Raitt. Fertility from AIM@SHAPE-VISIONAIR Shape Repository. Public domain Blub, Bob, and Ogre models by Keenan Crane. Open source Pawn model from OpenSubdiv.).

APPENDIX B
QUANTITATIVE EVALUATION

Smooth Surface Calculation Runtime by Mesh

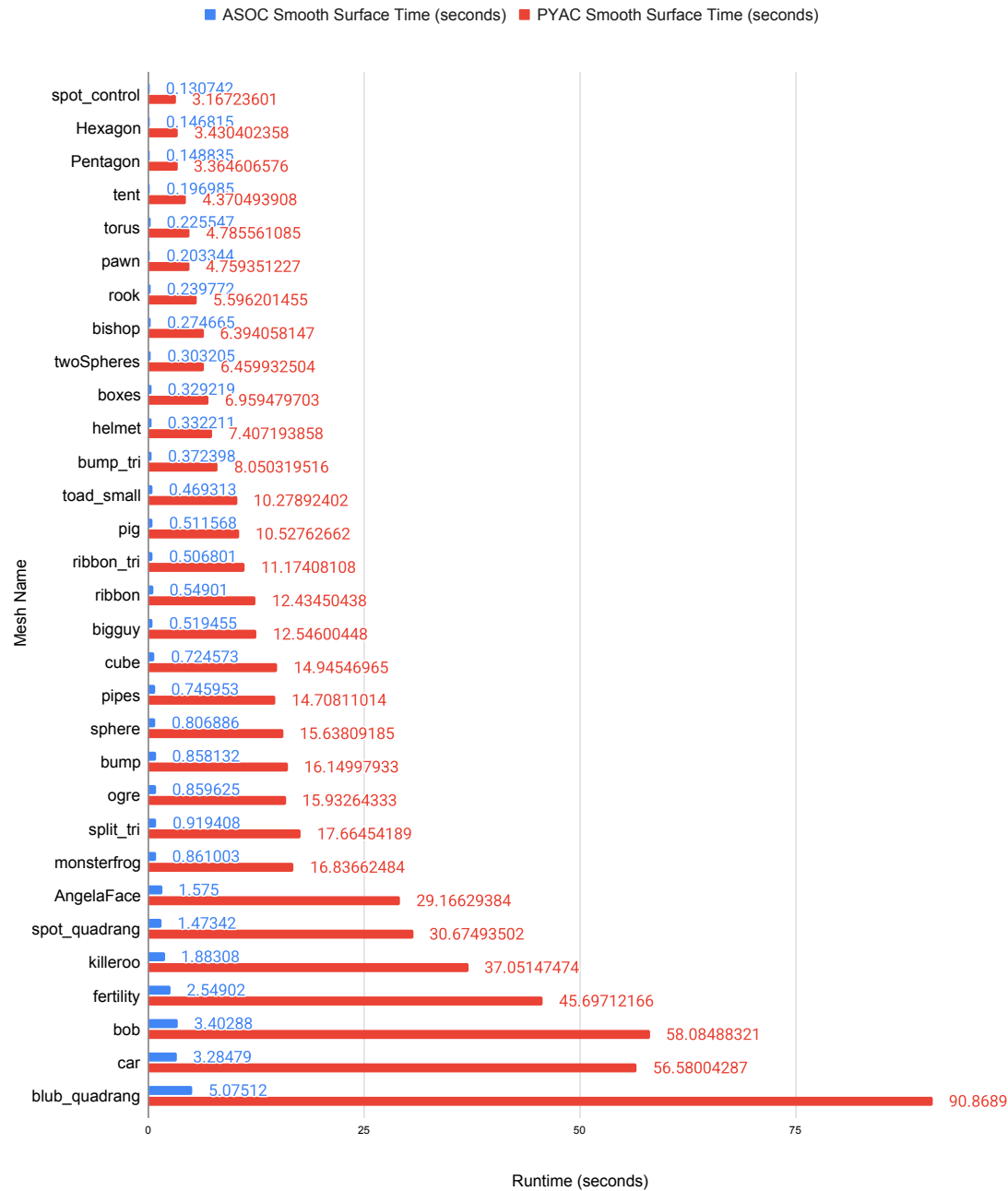


Figure B.1: Smooth surface runtime of ASOC (blue) versus PYAC (red) in seconds.

Contour Calculation Slowdown by Mesh

Including contour construction slowdown to show scale relative to other measurements.

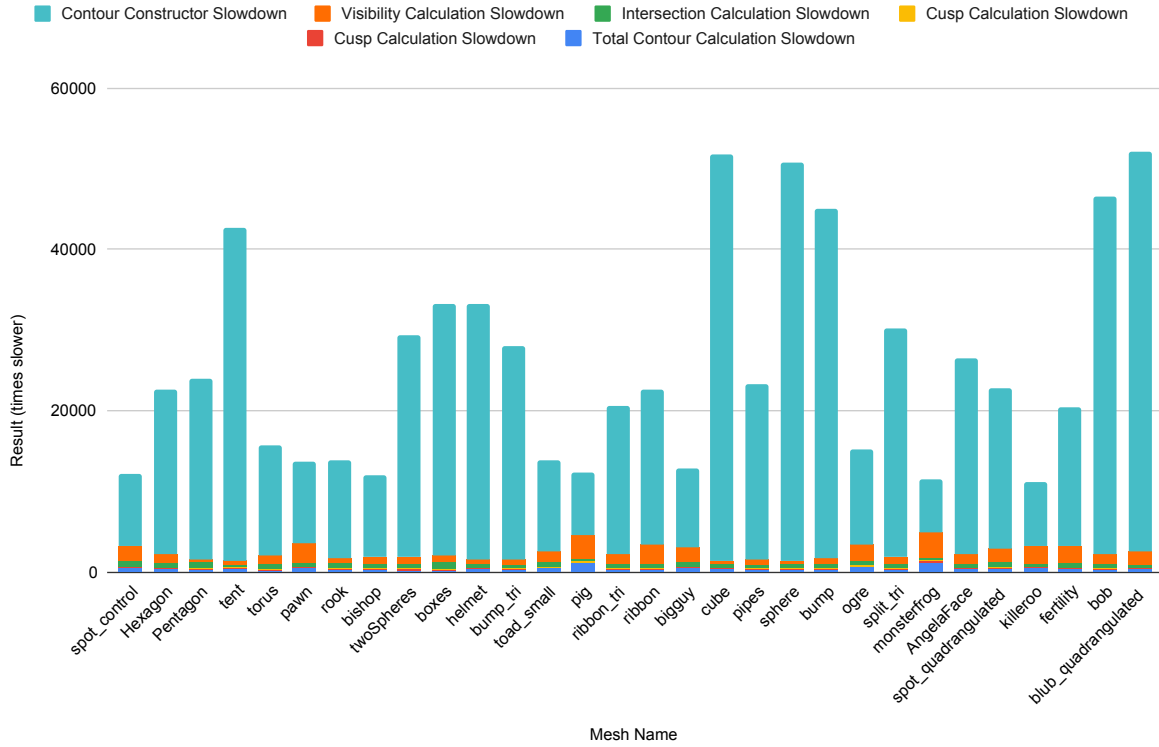


Figure B.2: Slowdown of each step within the occluding contour calculation itself with the occluding contour constructor call to showcase scale relative to the rest of the process.

Mesh Name	Mean	Maximum	Minimum	STDDEV
ASOC Faces (triangles, count)	4508	21290	816	4608
PYAC Faces (triangles, count)	4508	21290	816	4608
Segments (ASOC, count)	237	1561	3	397
Segments (PYAC, count)	815	6571	3	1581
Difference (Segments, count)	577.9	5010	0	1190
Intersection Calls (ASOC, count)	959	4586	0	1029
Intersection Calls (PYAC, count)	-	-	-	-
Difference (Intersection Calls, count)	-	-	-	-
Patches (ASOC, count)	50620	255500	9792	52690
Patches (PYAC, count)	50620	255500	9792	52690
Difference (Number of Patches)	0	0	0	0

Table B.1: Statistics for quantitative validity results. Note that these values do not include the failed "car" mesh in their calculation. Differences are calculated from the mean of differences across all 30 successful meshes out of the 31 total evaluated.

Contour Calculation Slowdown by Mesh

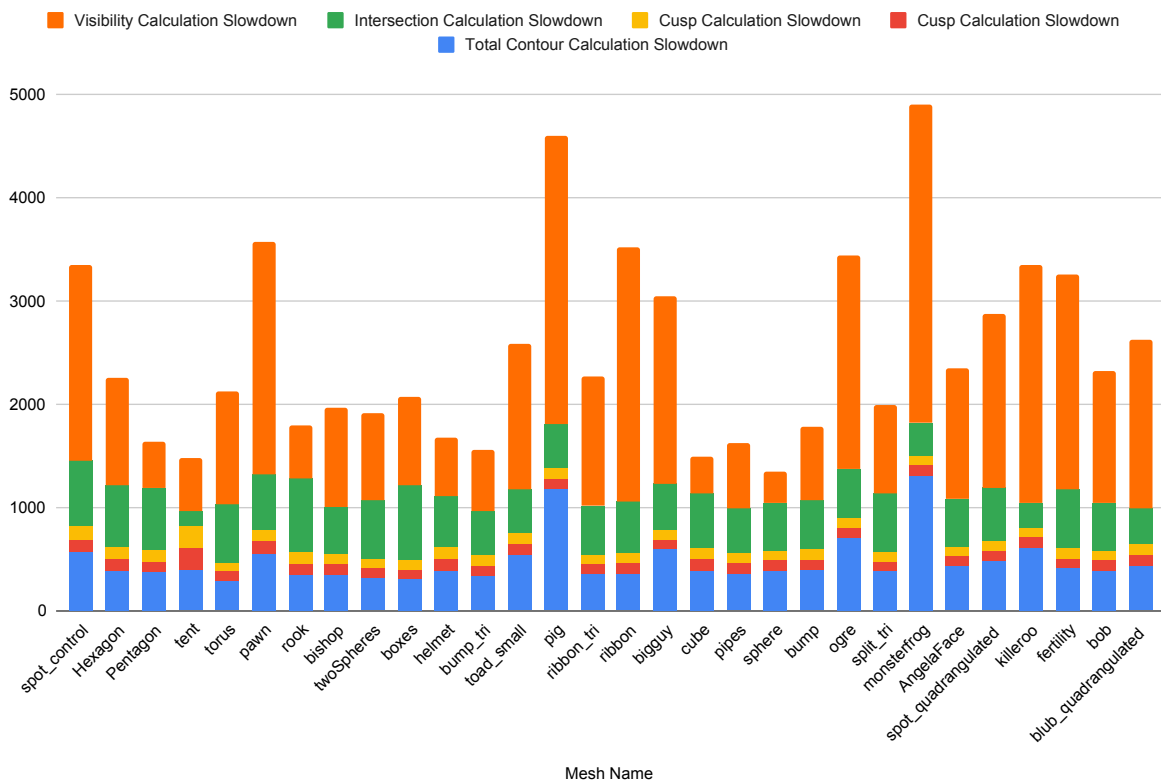


Figure B.3: Slowdown of each step within the occluding contour calculation itself without the occluding contour constructor call to showcase scale relative to the rest of the process. See Figure B.2 for the full figure including the occluding contour constructor call.

Statistic	Mean	Maximum	Minimum	STDDEV
AS Patch Boundary Edges (s)*	0.01212	0.04543	0.003424	0.009381
PY Patch Boundary Edges (s)*	0.1578	0.7231	0.03046	0.1604
Slowdown*	11.48	17.24	4.732	3.209
AS Vert. Normalization (s)*	0.00001450	0.00005300	0.00000500	0.0000105
PY Vert. Normalization (s)*	0.004679	0.02094	0.0008389	0.004706
Slowdown*	281.0	639.0	148.0	99.61
AS Vert. Transformation (s)	1.48E-5	5.30E-5	5.00E-6	1.06E-5
PY Vert. Transformation (s)	0.009293	0.03707	0.003514	0.007053
Slowdown	434.3	471.5	390.5	21.94
AS Smooth Surface (s)*	0.9832	5.075	0.1468	1.151
PY Smooth Surface (s)*	18.76	90.87	3.364	20.00
Slowdown*	20.84	24.15	17.07	2.081
AS Total Contour (s)	0.05301	0.2395	0.007105	0.05603
PY Total Contour (s)	31.62	210.5	2.699	46.67
Slowdown	480.6	1313	293.7	230.5
AS Cusp (s)	0.002196	0.009655	0.00004900	0.002094
PY Cusp (s)	0.2196	0.9673	0.01037	0.2078
Slowdown	105.1	211.6	89.87	21.71
AS Intersections (s)	0.003477	0.01642	0.00007600	0.004048
PY Intersections (s)	1.454	5.190	0.01089	1.267
Slowdown	484.9	720.5	143.3	122.2
AS Visibility (s)	0.007140	0.05972	0.00005100	0.01357
PY Visibility (s)	16.57	183.8	0.02301	38.64
Slowdown	1318	3077	296.8	771.2
AS Contour Constructor (s)	0.0009795	0.006728	0.00006900	0.001368
PY Contour Constructor (s)	15.04	60.64	2.675	14.66
Slowdown	23190	50240	6554	13820
AS Vector Write (s)	0.01383	0.06228	0.001389	0.01464
PY Vector Write (s)	0.1417	0.6633	0.01314	0.1487
Slowdown	10.25	10.90	9.373	0.3978
AS Raster Write (s)	0.3979	0.5023	0.3273	0.04829
PY Raster Write (s)	0.5488	1.160	0.3814	0.1831
Slowdown	1.382	2.687	0.8665	0.4267

Table B.2: Statistics table of runtime performance results throughout the end-to-end occluding contours pipeline. Note that slowdown is calculated from the mean of all slowdowns throughout the 30 successful meshes out of the 31 total evaluated. AS is shorthand for ASOC. PY is shorthand for PYAC. As denoted by the asterisk (*), values calculated utilizing the "car" mesh runtime results.

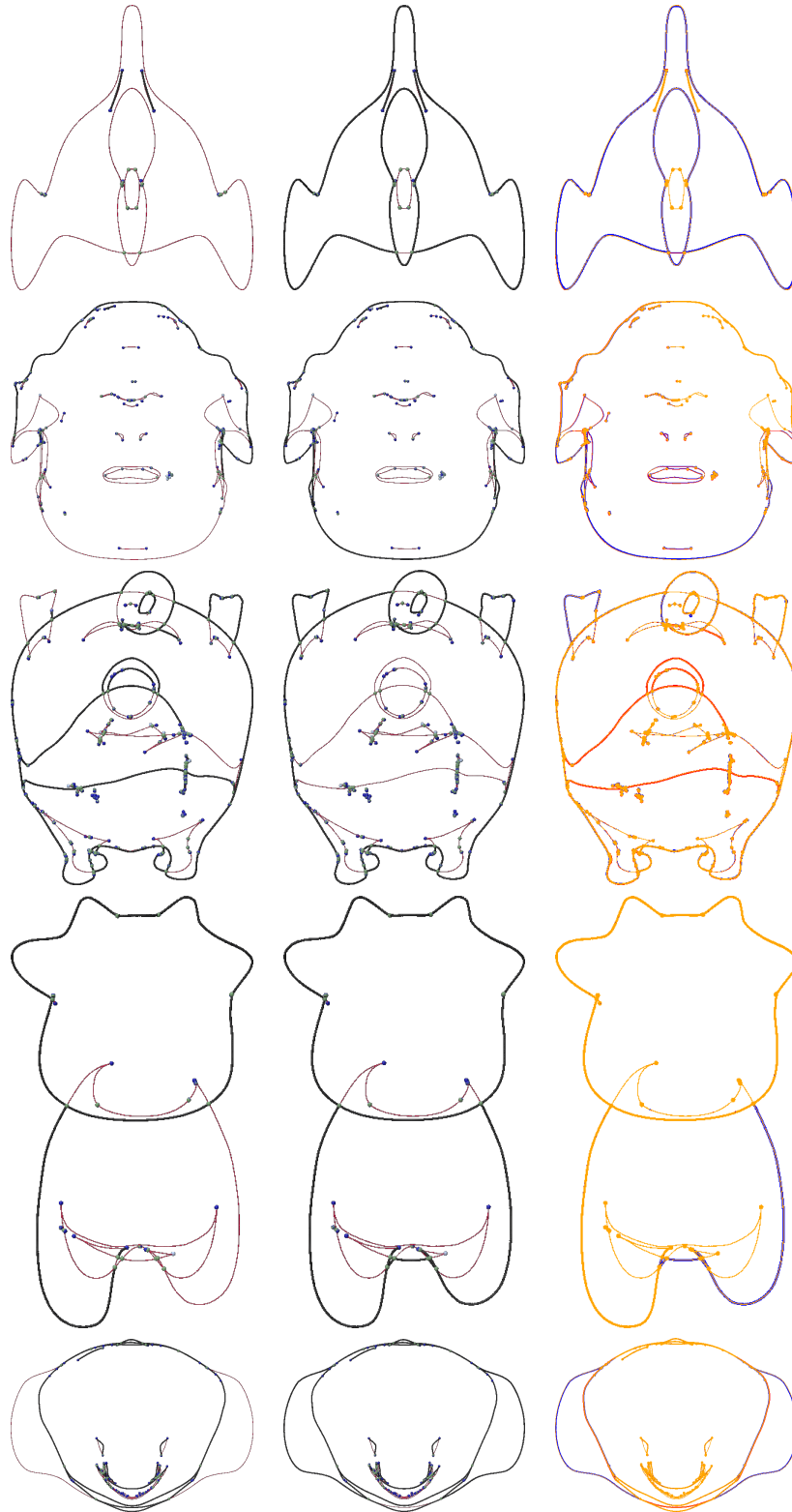


Table B.3: Raster full occluding contours showcasing ASOC's instability in its occluding contour visibility calculation using the propagation Quantitative Invisibility method (ASOC (left) vs PYAC (middle) vs difference (right)) (see Table A.1 for color map) (Public domain Bob, Blub, Spot, Pig, and Ogre models by Keenan Crane.).

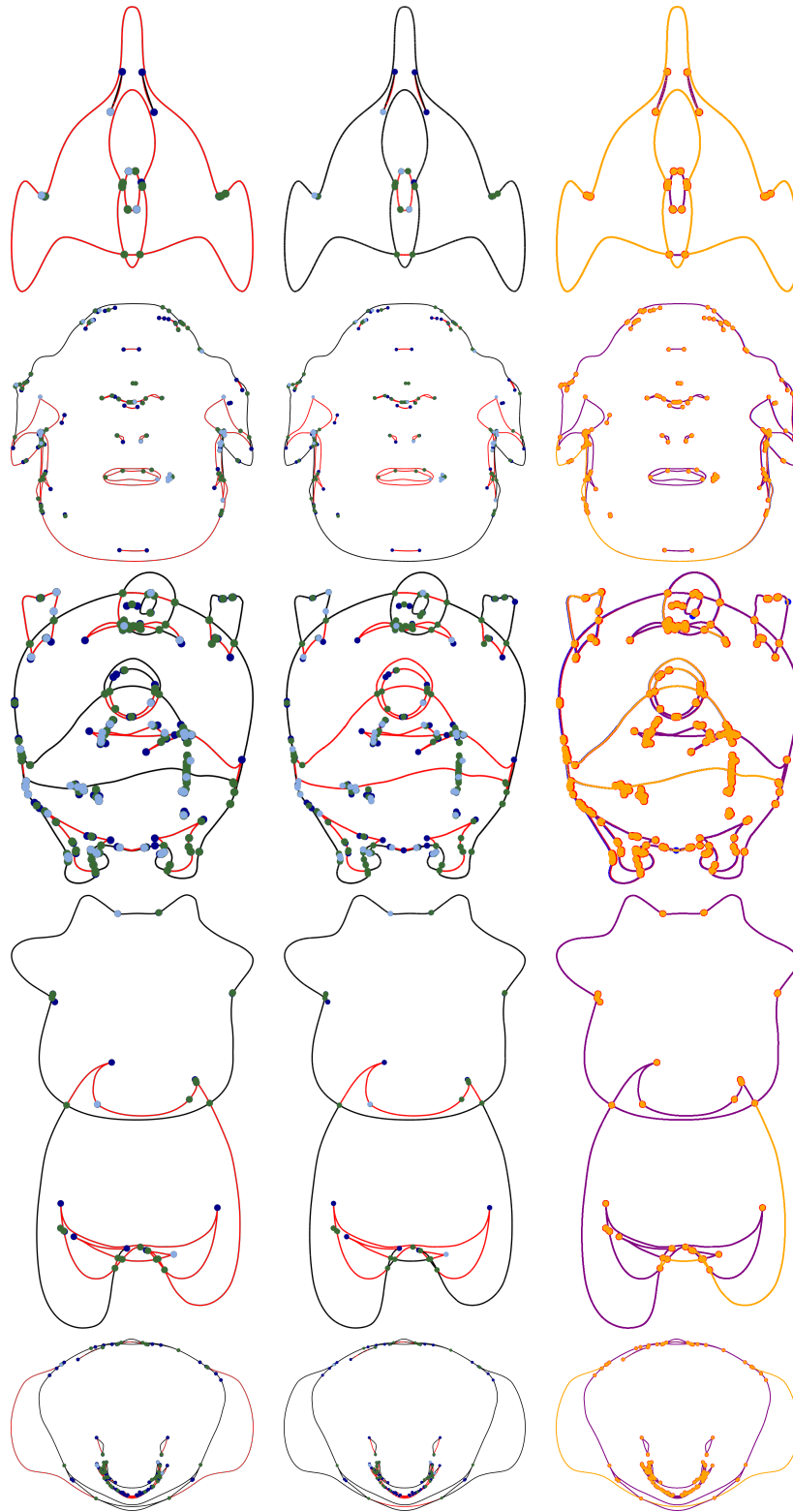


Table B.4: Vector full occluding contours showcasing ASOC’s instability in its occluding contour visibility calculation using propagation Quantitative Invisibility. (ASOC (left) vs PYAC (middle) vs difference (right)) (see Table A.1 for color map). (Public domain Bob, Blub, Spot, Pig, and Ogre models by Keenan Crane).

Mean Slowdown in Contour Calculation Pipeline

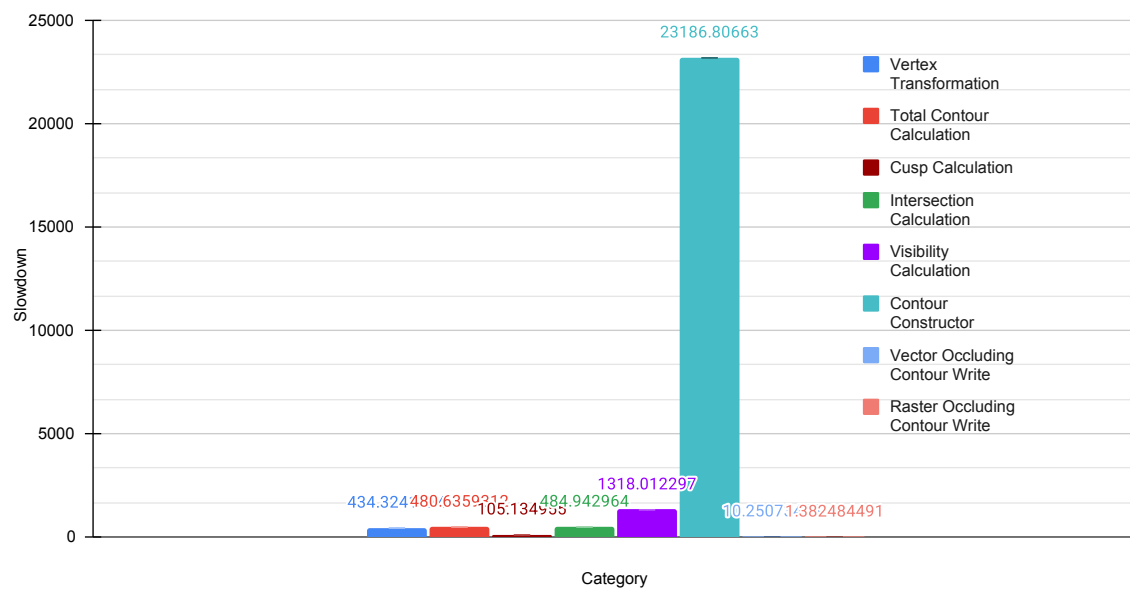


Figure B.4: Performance of various steps within the occluding contour calculation itself, with the occluding contour constructor call slowdown greatly outweighing the rest of the parts in the occluding contour calculation.

Number of Faces by Mesh

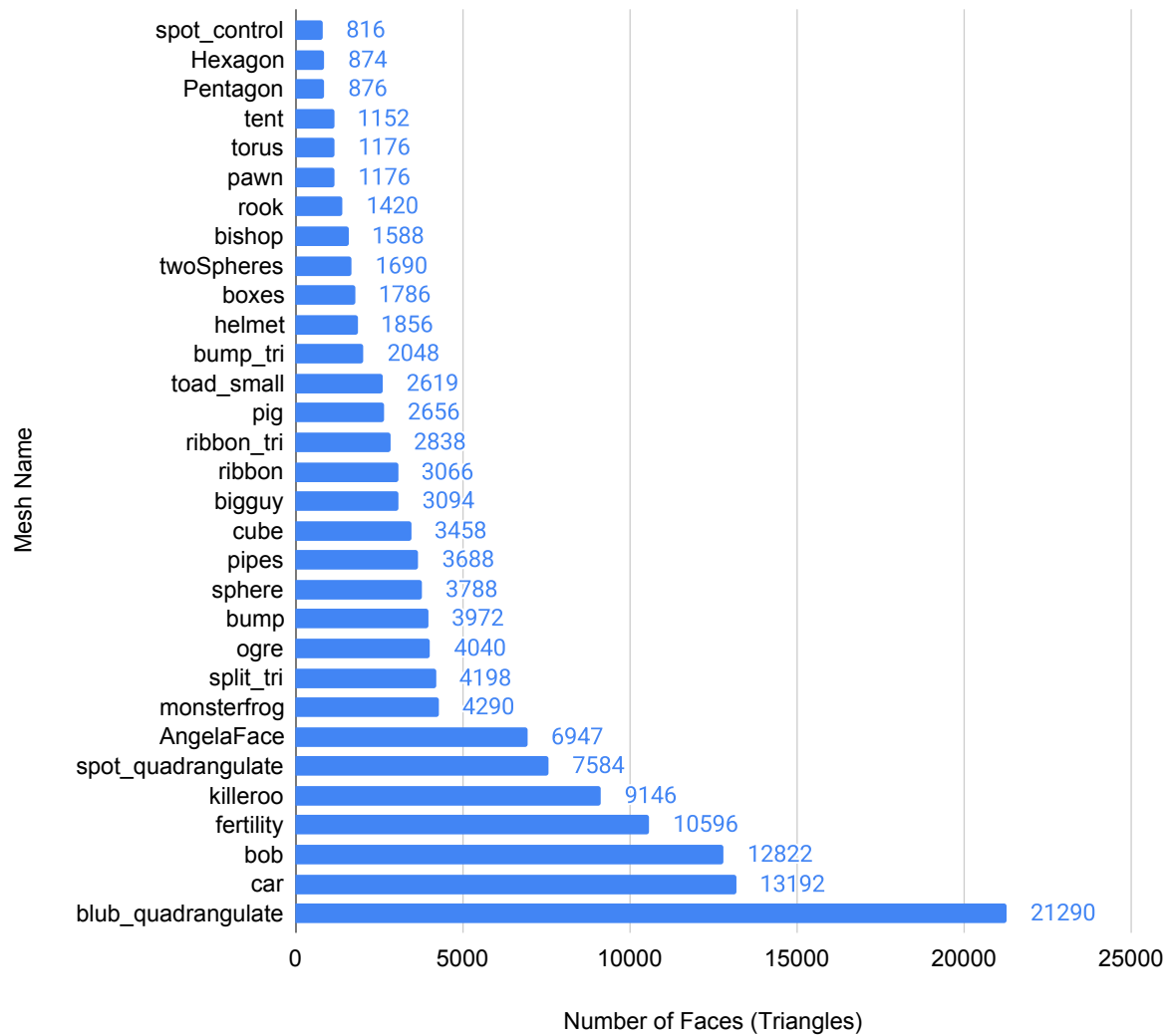


Figure B.5: Chart of the number of faces for all 31 meshes used for evaluation. Both ASOC and PYAC read in the same number of triangle faces. Note that all plots are sorted by increasing number of faces from smallest to largest.

ASOC Number of Intersection Calls by Mesh

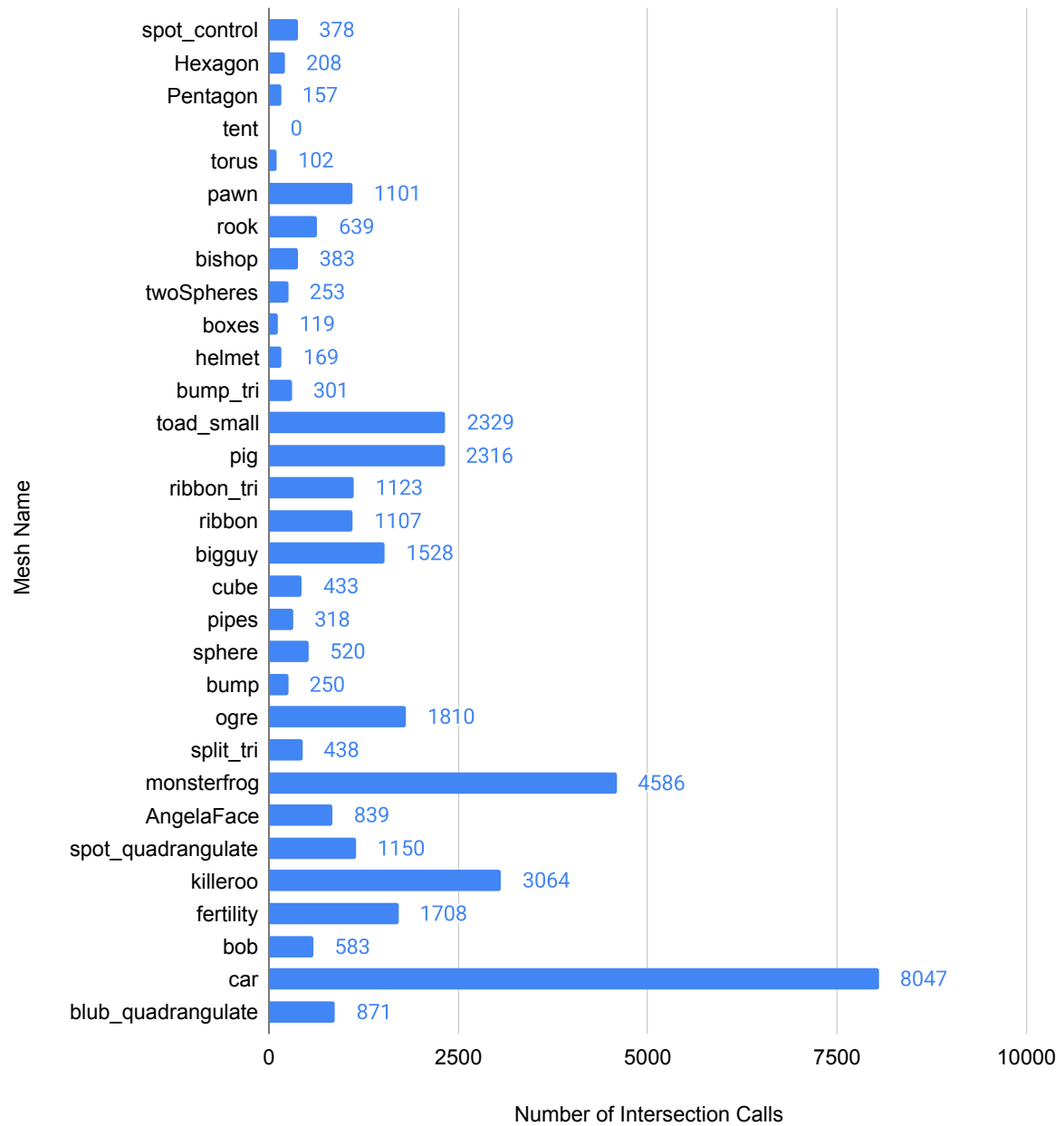


Figure B.6: Chart of the number of intersection calls by mesh for ASOC.

Number of Patches by Mesh

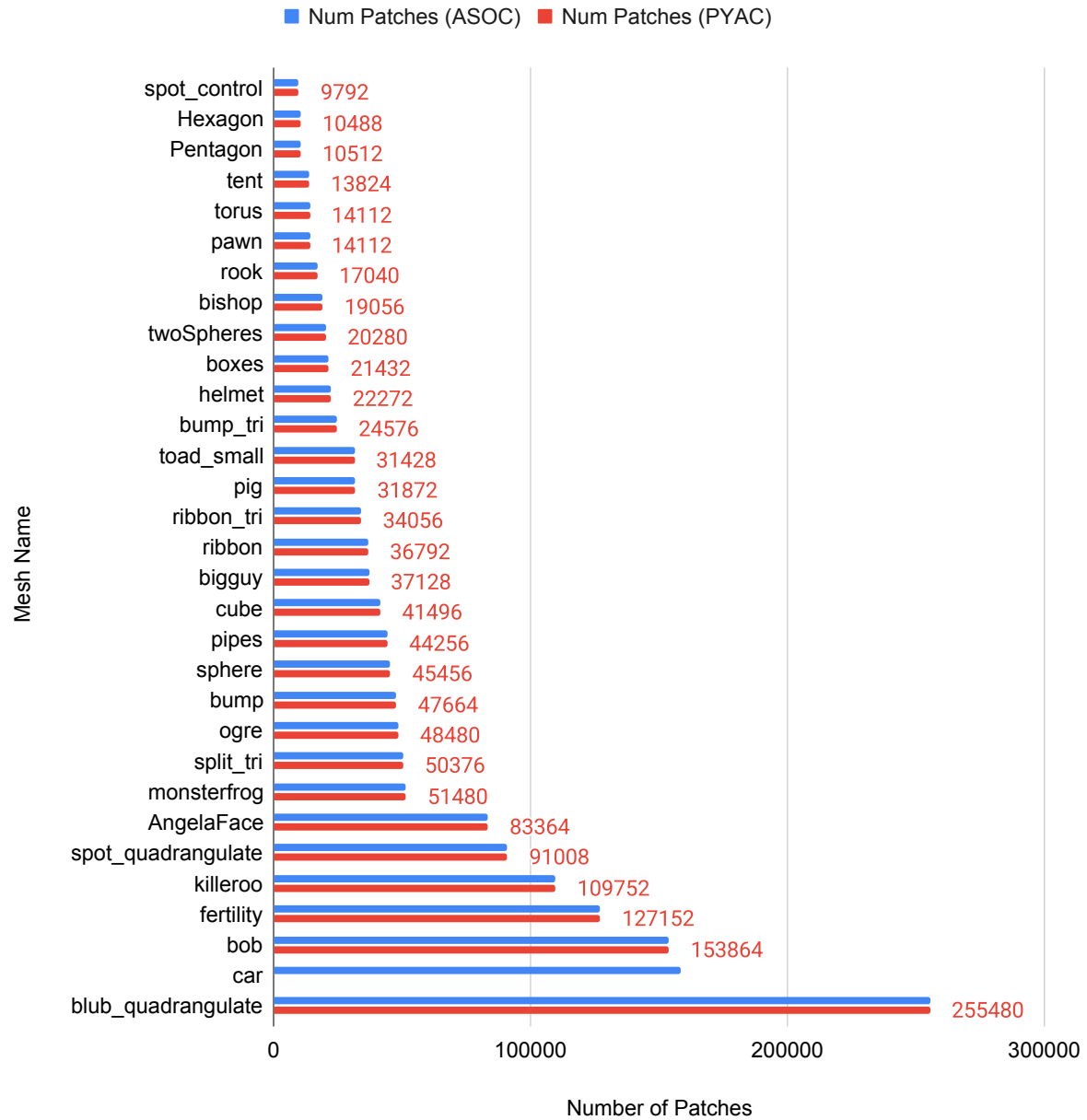


Figure B.7: Chart of the number of patches for the smooth surface of each mesh. Here, sameness in the number of patches represents the stability of the surface construction to noise. Note that the "car" mesh is missing its numerical data label for PYAC since its end-to-end occluding contour generation pipeline failed.

Number of Segments by Mesh

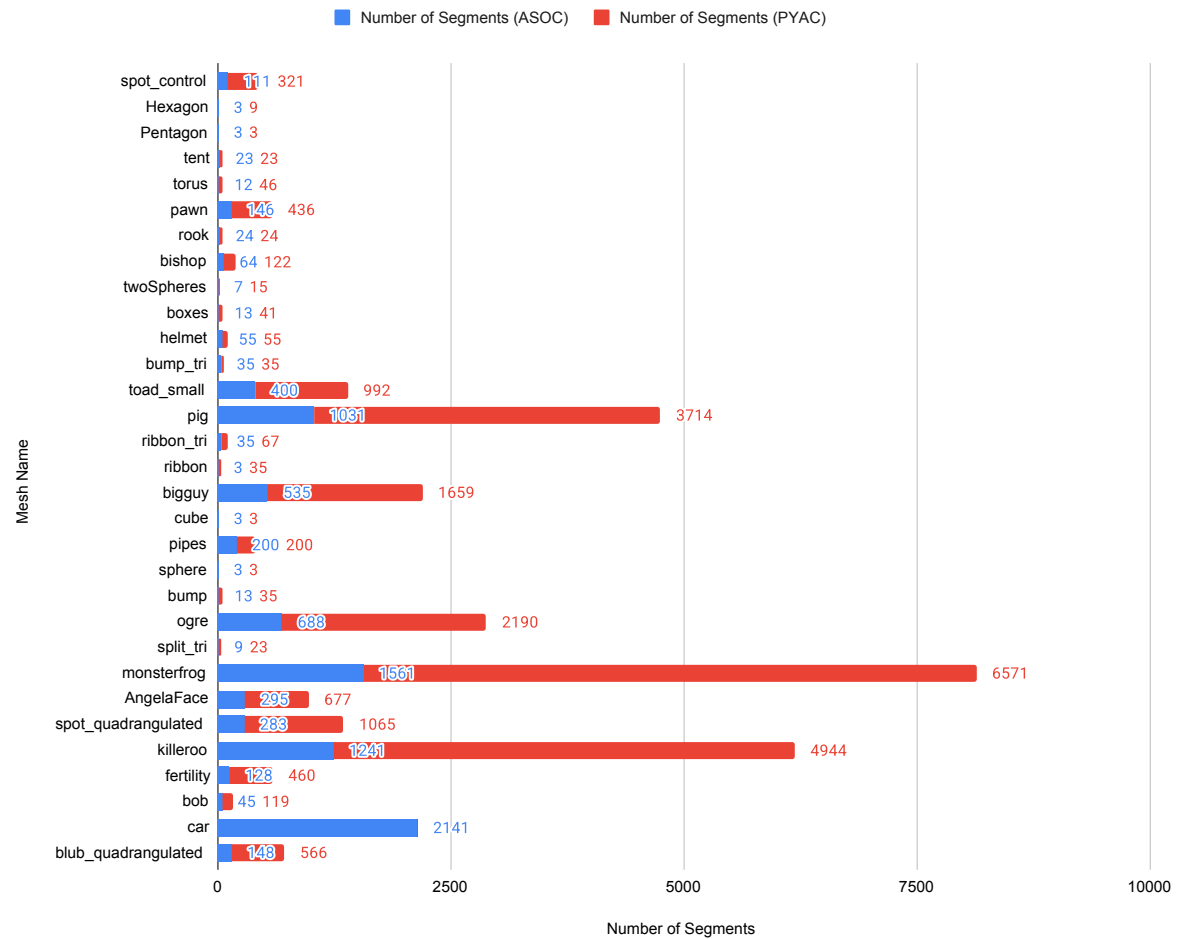


Figure B.8: Chart of the number of segments per mesh. Note that the "car" mesh does not have an equivalent PYAC bar data label because its evaluation failed the propagation Quantitative Invisiblity step.

Surface Calculation Slowdown by Mesh

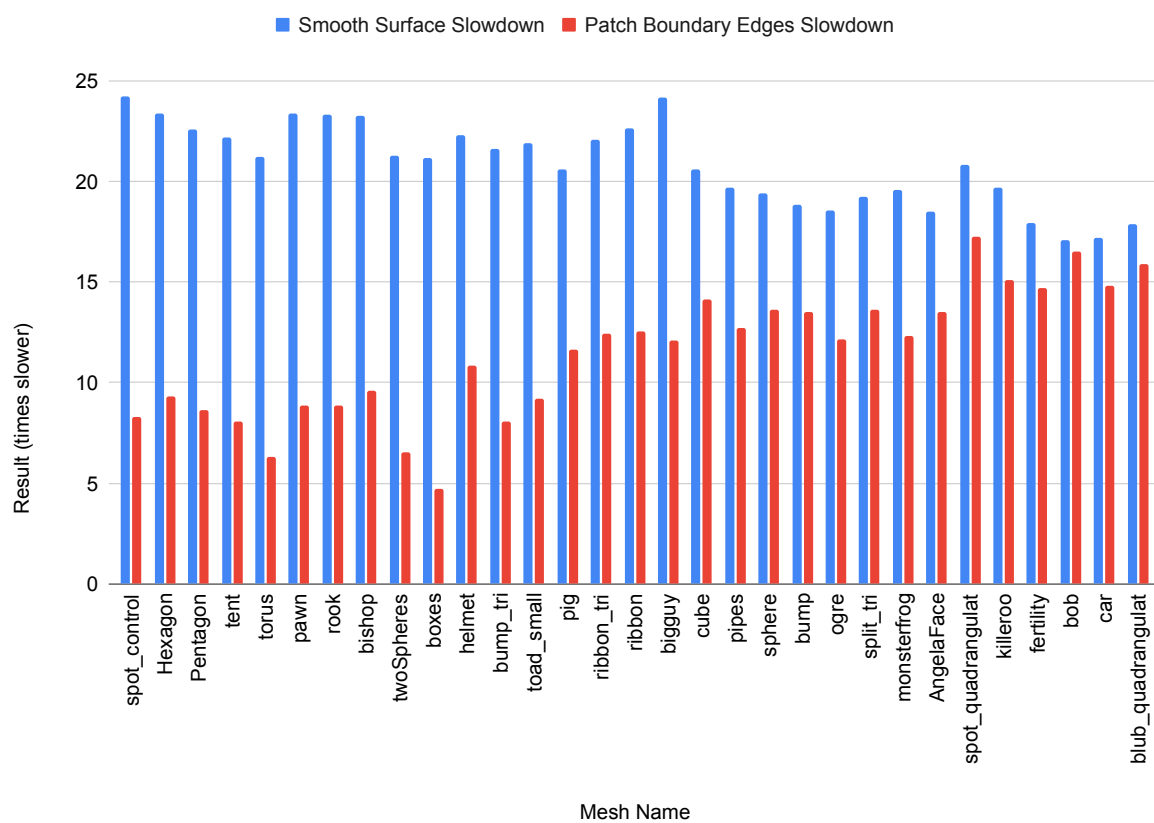


Figure B.9: Chart showing slowdown of PYAC smooth surface and patch boundary calculations.

ASOC Total Contour Calculation Time (seconds) vs Number of Faces (triangles)

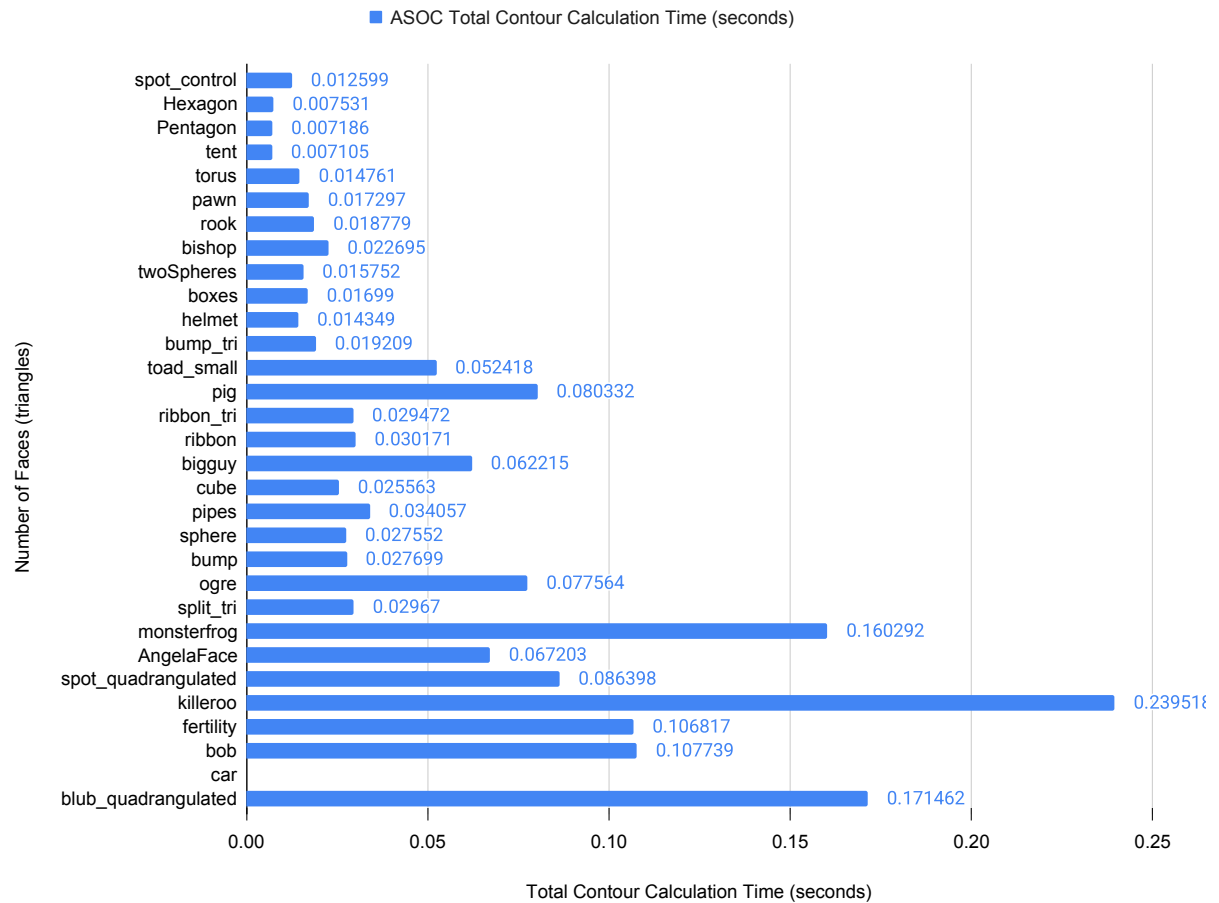


Figure B.10: ASOC total contour calculation time (seconds) vs. number of faces (triangles). Note that the "car" mesh does not have any metrics since it failed the propagation Quantitative Invisibility step.

PYAC Total Contour Calculation Time (seconds) vs Number of Faces (triangles)

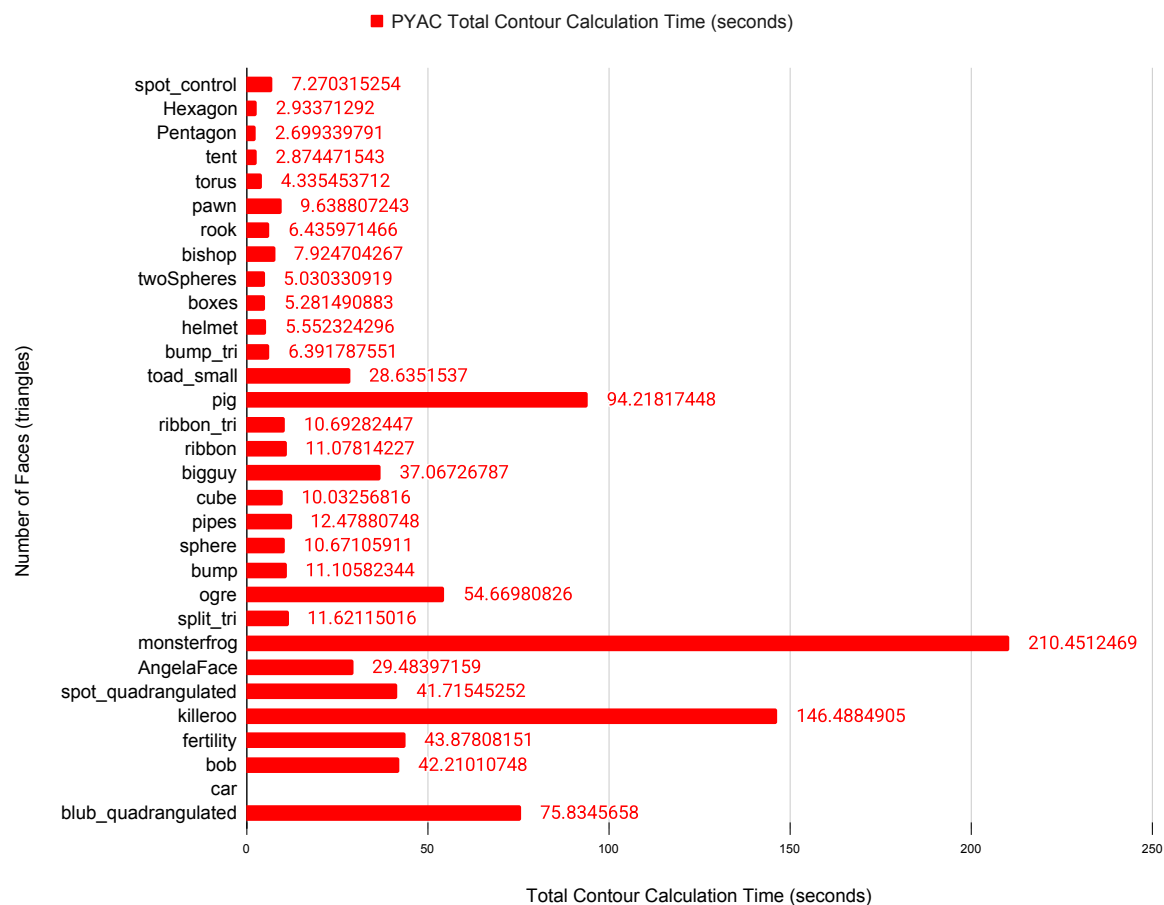


Figure B.11: PYAC total contour calculation time (seconds) vs. number of faces (triangles). Note that the "car" mesh does not have any metrics since it failed the propagation Quantitative Invisibility step.